

Amazon SDE1

Coding Interview Guide

73 problems: the 27 from your playlist, plus the 46 from the original list before the playlist link was shared. One page each. Brute force code AND optimal code, side by side.

A note on video transcripts: this session's network policy blocks youtube.com, and Claude in Chrome isn't connected in this cloud session either, so no transcripts could be pulled from the linked playlist. Every explanation below is written from general algorithmic knowledge, not transcribed from any video — see the per-problem note under each title, and the Sources page at the end. The 46 problems from the original (pre-playlist) list have no associated video at all, and their notes say so plainly.

Table of Contents

#	Problem	LC	Pattern
1	Two Sum	#1	Hash Map
2	Contains Duplicate	#217	Hash Set
3	Group Anagrams	#49	Hash Map
4	Product of Array Except Self	#238	Prefix/Suffix Products
5	Subarray Sum Equals K	#560	Prefix Sum + Hash Map
6	Valid Palindrome	#125	Two Pointers
7	Container With Most Water	#11	Two Pointers
8	Longest Substring Without Repeating Characters	#3	Sliding Window
9	Minimum Window Substring	#76	Sliding Window
10	Valid Parentheses	#20	Stack
11	Design Min Stack	#155	Stack (Auxiliary Min Stack)
12	Daily Temperatures	#739	Monotonic Stack
13	Evaluate Reverse Polish Notation	#150	Stack
14	Implement Queue using Stacks	#232	Stack (Two-Stack Trick)
15	Implement Stack using Queues	#225	Queue (Rotation Trick)
16	Maximum Depth of Binary Tree	#104	DFS / Recursion
17	Binary Tree Level Order Traversal	#102	BFS
18	Binary Tree Vertical Order Traversal	#314	BFS + Hash Map (Column Index)
19	Clone Graph	#133	DFS/BFS + Hash Map
20	Course Schedule	#207	Graph DFS / Topological Sort (Cycle Detection)
21	Binary Search	#704	Binary Search
22	Find Minimum in Rotated Sorted Array	#153	Binary Search (Modified)
23	LRU Cache	#146	Hash Map + Doubly Linked List
24	Random Pick with Weight	#528	Prefix Sum + Binary Search
25	Rotting Oranges	#994	Multi-Source BFS
26	Maximum Subarray	#53	Dynamic Programming (Kadane's Algorithm)
27	Top K Frequent Elements	#347	Bucket Sort (or Heap)
28	Maximum Product Subarray	#152	DP (Track Max & Min)
29	Find the Duplicate Number	#287	Fast & Slow Pointers
30	Single Number	#136	Bit Manipulation (XOR)
31	Move Zeroes	#283	Two Pointers (In-Place Partition)
32	Two Sum II - Input Array Is Sorted	#167	Two Pointers

33	Boats to Save People	#881	Two Pointers + Greedy
34	Minimum Size Subarray Sum	#209	Sliding Window (Variable Size)
35	Find All Anagrams in a String	#438	Sliding Window (Fixed Size)
36	Next Greater Element I	#496	Monotonic Stack
37	Decode String	#394	Stack
38	Simplify Path	#71	Stack
39	Find K Closest Elements	#658	Binary Search
40	Split Array Largest Sum	#410	Binary Search on the Answer
41	Pow(x, n)	#50	Fast Exponentiation (Divide & Conquer)
42	Add Two Numbers	#2	Linked List Traversal + Carry
43	Linked List Cycle	#141	Fast & Slow Pointers
44	Intersection of Two Linked Lists	#160	Two Pointers
45	Copy List with Random Pointer	#138	Hash Map (Clone Mapping)
46	Same Tree	#100	DFS Recursion
47	Validate Binary Search Tree	#98	DFS with Valid Range
48	Kth Smallest Element in a BST	#230	In-order Traversal
49	Construct Binary Tree from Preorder and Inorder Traversal	#105	DFS Recursion + Hash Map
50	Serialize and Deserialize Binary Tree	#297	DFS Preorder + Null Markers
51	Balanced Binary Tree	#110	Bottom-Up DFS
52	Subtree of Another Tree	#572	DFS Tree Matching
53	Walls and Gates	#286	Multi-Source BFS
54	Kth Largest Element in a Stream	#703	Heap (size-k min-heap)
55	Jump Game	#55	Greedy
56	Task Scheduler	#621	Greedy + Math
57	Car Pooling	#1094	Difference Array / Line Sweep
58	Generate Parentheses	#22	Backtracking
59	Combination Sum	#39	Backtracking
60	Letter Combinations of a Phone Number	#17	Backtracking
61	Valid Sudoku	#36	Hash Set Constraint Checking
62	Climbing Stairs	#70	DP (Fibonacci)
63	Perfect Squares	#279	DP (Unbounded Knapsack style)
64	Is Subsequence	#392	Two Pointers (Greedy Matching)
65	Distinct Subsequences	#115	2D DP (Counting)
66	Unique Binary Search Trees	#96	DP (Catalan Numbers)
67	Insert Delete GetRandom O(1)	#380	Hash Map + Array

68	Reverse Bits	#190	Bit Manipulation
69	Number of 1 Bits	#191	Bit Manipulation (Kernighan's)
70	Pascal's Triangle	#118	Simple DP (row from previous row)
71	Longest Common Prefix	#14	Vertical Scanning
72	Integer to Roman	#12	Greedy
73	Add Binary	#67	Two Pointers + Carry

Two Sum

LeetCode #1 | **EASY** | Pattern: Hash Map | Video: "Two Sum - Leetcode 1 - HashMap - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given an array of integers and a target, return the indices of the two numbers that add up to the target. Exactly one solution exists; can't use the same element twice. *Ask:* Exactly one solution guaranteed? Can't reuse the same element twice?

2. Brute force: Check every pair of indices and test if they sum to target.

```
def two_sum(nums, target):
    n = len(nums)
    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] + nums[j] == target:
                return [i, j]
    return []
```

3. Complexity: $O(n^2)$ time, $O(1)$ space

4. Bottleneck: Checking every pair re-scans the array for a partner I could just remember instead.

5. Optimize: Store each value's index as you scan; before adding, check if target minus current value was already seen — one pass instead of two nested loops.

6. Code it (narrate as you write):

```
def two_sum(nums, target):
    seen = {} # value -> index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

9. Final complexity: $O(n)$ time, $O(n)$ space

7. Test: nums=[2,7,11,15], target=9 -> map builds {2:0}, then 7's complement 2 is already in the map -> [0,1]. Edge: nums=[3,3], target=6 -> works since we check the map before inserting.

8. Other edge cases: no valid pair exists; negative numbers; target achieved by first two elements.

Memory hook: "I need the complement — remember what I've already seen."

Contains Duplicate

LeetCode #217 | **EASY** | Pattern: Hash Set | Video: "Contains Duplicate - Leetcode 217 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given an array of integers, return true if any value appears at least twice, and false if every element is distinct. Ask: Can the array be empty or have one element? Any bound on value range?

2. Brute force: Compare every pair of elements for equality.

```
def contains_duplicate(nums):
    n = len(nums)
    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] == nums[j]:
                return True
    return False
```

3. Complexity: $O(n^2)$ time, $O(1)$ space

4. Bottleneck: Comparing every pair re-checks values I could just remember in a set as I go.

5. Optimize: A hash set lets me check 'have I seen this before' in $O(1)$ average time, so one pass suffices.

6. Code it (narrate as you write):

```
def contains_duplicate(nums):
    seen = set()
    for num in nums:
        if num in seen:
            return True
        seen.add(num)
    return False

# One-liner alt: len(set(nums)) != len(nums)
```

9. Final complexity: $O(n)$ time, $O(n)$ space

7. Test: nums=[1,2,3,1] -> seen builds {1,2,3}, hits 1 again -> True. Edge: empty array -> loop never runs, False.

8. Other edge cases: all unique; all identical; single element.

Memory hook: "Seen it before? Set tells you instantly."

Group Anagrams

LeetCode #49 | **MEDIUM** | Pattern: Hash Map | Video: "Group Anagrams - Categorize Strings by Count - Leetcode 49" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given an array of strings, group the anagrams together (any order). Two strings are anagrams if they contain the same letters with the same counts. Ask: Case-sensitive? Only lowercase letters guaranteed, or could there be other characters?

2. Brute force: Compare every string against every other by sorting both and checking equality.

```
def group_anagrams(strs):
    groups = []
    used = [False] * len(strs)
    for i in range(len(strs)):
        if used[i]:
            continue
        group = [strs[i]]
        for j in range(i + 1, len(strs)):
            if not used[j] and sorted(strs[i]) == sorted(strs[j]):
                group.append(strs[j])
                used[j] = True
        groups.append(group)
    return groups
```

3. Complexity: $O(n^2 * k \log k)$ time, $O(n * k)$ space

4. Bottleneck: Sorting each string to compare it against every other string is repeated, wasted work.

5. Optimize: A 26-count tuple (or sorted string) is identical for all anagrams of a word, so it works as a hash map key that groups them in one pass.

6. Code it (narrate as you write):

```
from collections import defaultdict

def group_anagrams(strs):
    groups = defaultdict(list)
    for word in strs:
        count = [0] * 26
        for ch in word:
            count[ord(ch) - ord('a')] += 1
        groups[tuple(count)].append(word)
    return list(groups.values())
```

9. Final complexity: $O(n * k)$ time, $O(n * k)$ space

7. Test: ['eat','tea','tan'] -> 'eat'/'tea' share a count key, 'tan' gets its own. Edge: empty string -> its own group.

8. Other edge cases: single string in the list; all strings anagrams of each other; duplicate identical strings.

Memory hook: "Same letter counts = same key = same bucket."

Product of Array Except Self

LeetCode #238 | **MEDIUM** | Pattern: Prefix/Suffix Products | Video: "Product of Array Except Self - Leetcode 238 - Python" — NeetCode.

Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given an array, return an array where each element is the product of all other elements, without using division, in $O(n)$. Ask: Can the array contain zero(s)? Division truly disallowed?

2. Brute force: For each index, loop over the rest of the array and multiply everything else.

```
def product_except_self(nums):
    n = len(nums)
    result = []
    for i in range(n):
        product = 1
        for j in range(n):
            if j != i:
                product *= nums[j]
        result.append(product)
    return result
```

3. Complexity: $O(n^2)$ time, $O(1)$ extra space

4. Bottleneck: Re-multiplying the whole array minus one element, for every element, redoes overlapping work.

5. Optimize: Answer at i = (product of everything left of i) * (product of everything right of i). Build prefix products left-to-right, then fold in suffix products right-to-left.

6. Code it (narrate as you write):

```
def product_except_self(nums):
    n = len(nums)
    result = [1] * n
    prefix = 1
    for i in range(n):
        result[i] = prefix
        prefix *= nums[i]
    suffix = 1
    for i in range(n - 1, -1, -1):
        result[i] *= suffix
        suffix *= nums[i]
    return result
```

9. Final complexity: $O(n)$ time, $O(1)$ extra space

7. Test: [1,2,3,4] -> prefix pass gives [1,1,2,6], suffix pass folds in to [24,12,8,6]. Edge: one zero, e.g. [1,2,0,4] -> [0,0,8,0].

8. Other edge cases: two or more zeros (all outputs 0); negative numbers; array of length 2.

Memory hook: "Everything to my left, times everything to my right."

Subarray Sum Equals K

LeetCode #560 | **MEDIUM** | Pattern: Prefix Sum + Hash Map | Video: "Subarray Sum Equals K - Prefix Sums - Leetcode 560 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given an array of integers and an integer k, return the total number of contiguous subarrays whose sum equals k. Numbers can be negative. Ask: Can values be negative? Does an empty subarray count (no, subarrays are non-empty contiguous runs)?

2. Brute force: Check every subarray's sum directly (or extend a running sum per start index).

```
def subarray_sum(nums, k):
    count = 0
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            if total == k:
                count += 1
    return count
```

3. Complexity: $O(n^2)$ time, $O(1)$ space

4. Bottleneck: Recomputing each subarray's sum from scratch re-adds numbers I've already summed before.

5. Optimize: $\text{sum}(i..j) = \text{prefix}[j] - \text{prefix}[i-1]$. So $\text{sum}(i..j) == k$ means $\text{prefix}[i-1] == \text{prefix}[j] - k$. Track how many times each prefix sum has occurred so far.

6. Code it (narrate as you write):

```
from collections import defaultdict

def subarray_sum(nums, k):
    count = 0
    prefix_sum = 0
    seen = defaultdict(int)
    seen[0] = 1 # empty prefix
    for num in nums:
        prefix_sum += num
        count += seen[prefix_sum - k]
        seen[prefix_sum] += 1
    return count
```

9. Final complexity: $O(n)$ time, $O(n)$ space

7. Test: [1,1,1], k=2 -> prefix sums 1,2,3; seen[0]=1 upfront; count becomes 2. Edge: k=0 with a zero in the array needs seen[0]=1 handled correctly.

8. Other edge cases: all negative numbers; k not achievable at all; single-element array equal to k.

Memory hook: "Two equal prefix sums — the gap between them sums to k."

Valid Palindrome

LeetCode #125 | **EASY** | Pattern: Two Pointers | Video: "Valid Palindrome - Leetcode 125 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given a string, return true if it's a palindrome after converting to lowercase and removing all non-alphanumeric characters. *Ask:* Only alphanumeric characters count, case-insensitive? Can the string be empty?

2. Brute force: Build a cleaned version of the string, then compare it to its reverse.

```
def is_palindrome(s):
    cleaned = [c.lower() for c in s if c.isalnum()]
    return cleaned == cleaned[::-1]
```

3. Complexity: $O(n)$ time, $O(n)$ space

4. Bottleneck: Building a whole cleaned copy of the string uses $O(n)$ extra space I don't need.

5. Optimize: Use two pointers from both ends, skipping non-alphanumeric characters in place, comparing directly — no extra string built.

6. Code it (narrate as you write):

```
def is_palindrome(s):
    left, right = 0, len(s) - 1
    while left < right:
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1
        if s[left].lower() != s[right].lower():
            return False
        left += 1
        right -= 1
    return True
```

9. Final complexity: $O(n)$ time, $O(1)$ space

7. Test: "A man, a plan, a canal: Panama" -> pointers skip punctuation/case, converge to True. Edge: string of only punctuation -> pointers cross immediately, True.

8. Other edge cases: single character; numbers mixed with letters; already-lowercase palindrome.

Memory hook: "Skip junk, compare from both ends inward."

Container With Most Water

LeetCode #11 | **MEDIUM** | Pattern: Two Pointers | Video: "Container with Most Water - Leetcode 11 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given heights of vertical lines, find two lines that together with the x-axis form a container holding the most water. Return the max area. Ask: Can heights be zero? At least two lines guaranteed?

2. Brute force: Check every pair of lines and compute the area they'd contain.

```
def max_area(height):
    best = 0
    n = len(height)
    for i in range(n):
        for j in range(i + 1, n):
            area = (j - i) * min(height[i], height[j])
            best = max(best, area)
    return best
```

3. Complexity: $O(n^2)$ time, $O(1)$ space

4. Bottleneck: Checking every pair of lines recomputes areas for pairs that can't possibly beat the current best.

5. Optimize: Start with the widest container (both ends). The shorter wall is always the bottleneck, so move it inward — moving the taller wall could never help.

6. Code it (narrate as you write):

```
def max_area(height):
    left, right = 0, len(height) - 1
    best = 0
    while left < right:
        h = min(height[left], height[right])
        best = max(best, (right - left) * h)
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1
    return best
```

9. Final complexity: $O(n)$ time, $O(1)$ space

7. Test: [1,8,6,2,5,4,8,3,7] -> pointers start at ends (area $7*1=7$), move shorter side inward, converge to $best=49$. Edge: all equal heights -> widest pair (the ends) is optimal immediately.

8. Other edge cases: two lines only; one very tall line surrounded by short ones; strictly increasing heights.

Memory hook: "Move the shorter wall — the taller one was never the limit."

Longest Substring Without Repeating Characters

LeetCode #3 | **MEDIUM** | Pattern: Sliding Window | Video: "Longest Substring Without Repeating Characters - Leetcode 3 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given a string, find the length of the longest substring without repeating characters. Ask: Empty string possible? Only ASCII, or could there be unicode?

2. Brute force: Check every substring for uniqueness directly.

```
def length_of_longest_substring(s):
    best = 0
    for i in range(len(s)):
        seen = set()
        for j in range(i, len(s)):
            if s[j] in seen:
                break
            seen.add(s[j])
            best = max(best, j - i + 1)
    return best
```

3. Complexity: $O(n^2)$ time, $O(\min(n, \text{charset}))$ space

4. Bottleneck: Restarting a fresh scan from every index re-examines characters already confirmed unique.

5. Optimize: Expand the window right; if the new character is already in the window, shrink from the left until the duplicate is gone — window never revisits a position twice.

6. Code it (narrate as you write):

```
def length_of_longest_substring(s):
    seen = set()
    left = 0
    best = 0
    for right in range(len(s)):
        while s[right] in seen:
            seen.remove(s[left])
            left += 1
        seen.add(s[right])
        best = max(best, right - left + 1)
    return best
```

9. Final complexity: $O(n)$ time, $O(\min(n, \text{charset}))$ space

7. Test: "abcabcbb" -> window grows to "abc" (len 3), hits repeat 'a', shrinks left, best stays 3. Edge: "" -> loop never runs, best=0.

8. Other edge cases: all identical characters ("aaaa"); string with no repeats at all; single character.

Memory hook: "Duplicate found? Shrink from the left until it's gone."

Minimum Window Substring

LeetCode #76 | **HARD** | Pattern: Sliding Window | Video: "Minimum Window Substring - Airbnb Interview Question - Leetcode 76" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given strings s and t , find the minimum window substring of s that contains every character of t (with at least its multiplicity), or empty string if none exists. Ask: Multiplicity of characters in t matters (need at least that many of each)? What if t is empty or longer than s ?

2. Brute force: Check every substring of s and test whether it contains all of t 's characters.

```
from collections import Counter

def min_window(s, t):
    need = Counter(t)
    best = ""
    for i in range(len(s)):
        for j in range(i, len(s)):
            window = Counter(s[i:j+1])
            if all(window[c] >= need[c] for c in need):
                if best == "" or j - i + 1 < len(best):
                    best = s[i:j+1]
    return best
```

3. Complexity: $O(n^3)$ time, $O(n)$ space

4. Bottleneck: Checking every substring directly recomputes its character counts from scratch each time.

5. Optimize: Expand right until the window satisfies all of t 's counts; then shrink from the left as much as possible while still valid, recording the smallest valid window found.

6. Code it (narrate as you write):

```
from collections import Counter

def min_window(s, t):
    if not t:
        return ""
    need = Counter(t)
    missing = len(t)
    left = 0
    best = (float('inf'), 0, 0)
    for right, ch in enumerate(s):
        if need[ch] > 0:
            missing -= 1
            need[ch] -= 1
        while missing == 0:
            if right - left + 1 < best[0]:
                best = (right - left + 1, left, right)
            need[s[left]] += 1
            if need[s[left]] > 0:
                missing += 1
            left += 1
    return "" if best[0] == float('inf') else s[best[1]:best[2]+1]
```

9. Final complexity: $O(n + m)$ time, $O(\text{charset})$ space

7. Test: $s = \text{"ADOBECODEBANC"} , t = \text{"ABC"} \rightarrow$ window expands/shrinks to "BANC". Edge: t empty $\rightarrow$ return "" immediately.

8. Other edge cases: t longer than s (no answer, return ""); t 's characters not present in s at all; s equals t exactly.

Memory hook: "Expand until valid, then shrink until it breaks — record the smallest valid window."

Valid Parentheses

LeetCode #20 | **EASY** | Pattern: Stack | Video: "Valid Parentheses - Stack - Leetcode 20 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given a string of just '{}[]()' characters, determine if the brackets are validly matched and nested. Ask: Only the six bracket characters appear, no other text mixed in?

2. Brute force: Repeatedly remove any adjacent matched pair like '()', '[]', '{}'. '[]' from the string until no more removals are possible; valid iff the string becomes empty.

```
def is_valid(s):
    pairs = ["()", "[]", "{}"]
    changed = True
    while changed:
        changed = False
        for p in pairs:
            if p in s:
                s = s.replace(p, "", 1)
                changed = True
    return len(s) == 0
```

3. Complexity: $O(n^2)$ time, $O(n)$ space

4. Bottleneck: Repeatedly scanning and removing matched pairs re-scans the whole string over and over.

5. Optimize: Push opening brackets. On a closing bracket, it must match the top of the stack (the most recently opened, still-unclosed bracket); if not, invalid.

6. Code it (narrate as you write):

```
def is_valid(s):
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}'}
    for ch in s:
        if ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
        else:
            stack.append(ch)
    return not stack
```

9. Final complexity: $O(n)$ time, $O(n)$ space

7. Test: "{}[]()" -> every open matches the most recent close, stack empties, True. Edge: "()" -> first char is a close with an empty stack, False.

8. Other edge cases: single unmatched bracket; only opening brackets; only closing brackets; empty string (trivially valid).

Memory hook: "Push opens; a close must match the most recent open."

Design Min Stack

LeetCode #155 | **MEDIUM** | Pattern: Stack (Auxiliary Min Stack) | Video: "Design Min Stack - Amazon Interview Question - Leetcode 155 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Design a stack supporting push, pop, top, and getMin, all in $O(1)$ time. Ask: Do push/pop/top/getMin ALL need true $O(1)$, not just amortized? Duplicate values possible?

2. Brute force: Plain stack; getMin scans the whole stack every time it's called.

```
class MinStack:
    def __init__(self):
        self.stack = []
    def push(self, val):
        self.stack.append(val)
    def pop(self):
        self.stack.pop()
    def top(self):
        return self.stack[-1]
    def get_min(self):
        return min(self.stack)  #  $O(n)$  !
```

3. Complexity: push/pop/top $O(1)$, getMin $O(n)$ time, $O(n)$ space

4. Bottleneck: Scanning the whole stack on every getMin call redoes a search I could track incrementally instead.

5. Optimize: Maintain a second stack that always stores the running minimum alongside each push, so it's popped in lockstep and always instantly available.

6. Code it (narrate as you write):

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []
    def push(self, val):
        self.stack.append(val)
        m = val if not self.min_stack else min(val, self.min_stack[-1])
        self.min_stack.append(m)
    def pop(self):
        self.stack.pop()
        self.min_stack.pop()
    def top(self):
        return self.stack[-1]
    def get_min(self):
        return self.min_stack[-1]
```

9. Final complexity: $O(1)$ all ops time, $O(n)$ space

7. Test: push(-2), push(0), push(-3), getMin()->-3, pop(), top()->0, getMin()->-2. Edge: push same min twice then pop once -> getMin still correct via $\leq$ comparison.

8. Other edge cases: popping down to empty; strictly increasing pushes; strictly decreasing pushes.

Memory hook: "A second stack, always remembering the min-so-far."

Daily Temperatures

LeetCode #739 | **MEDIUM** | Pattern: Monotonic Stack | Video: "Daily Temperatures - Monotonic Stack - Leetcode 739 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given daily temperatures, return an array where `answer[i]` is how many days until a warmer temperature; 0 if none exists. Ask: Return 0 when no warmer day exists? Temperatures guaranteed positive integers?

2. Brute force: For each day, scan forward until a warmer day is found.

```
def daily_temperatures(temps):
    n = len(temps)
    answer = [0] * n
    for i in range(n):
        for j in range(i + 1, n):
            if temps[j] > temps[i]:
                answer[i] = j - i
                break
    return answer
```

3. Complexity: $O(n^2)$ time, $O(1)$ extra space

4. Bottleneck: Scanning forward from every day re-examines days already ruled out as 'not warmer' for a previous day.

5. Optimize: Keep a stack of day-indices whose warmer day hasn't been found yet. When a warmer day appears, it resolves every cooler day still waiting on the stack.

6. Code it (narrate as you write):

```
def daily_temperatures(temps):
    answer = [0] * len(temps)
    stack = [] # indices, decreasing temps
    for i, temp in enumerate(temps):
        while stack and temps[stack[-1]] < temp:
            j = stack.pop()
            answer[j] = i - j
        stack.append(i)
    return answer
```

9. Final complexity: $O(n)$ time, $O(n)$ space

7. Test: [73,74,75,71,69,72,76,73] -> stack resolves each day as a warmer one appears; day 0 waits 1 day. Edge: strictly decreasing temps -> nothing ever resolves, all zeros.

8. Other edge cases: all identical temperatures (never strictly warmer, all zeros); single day; strictly increasing temps.

Memory hook: "Stack of unresolved days; a warmer day resolves everyone smaller beneath it."

Evaluate Reverse Polish Notation

LeetCode #150 | **MEDIUM** | Pattern: Stack | Video: "Evaluate Reverse Polish Notation - Leetcode 150 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given tokens of a postfix arithmetic expression (+ - * /), evaluate it and return the integer result. Ask: Division truncates toward zero, not floor? Expression always guaranteed valid?

2. Brute force: There's no simpler alternative to a stack here; a naive idea (convert to infix and use a general parser) is more complex, not simpler.

```
# A general infix-parser approach still needs precedence
# handling and is strictly more complex than the stack
# approach below for postfix notation - not shown as a
# separate 'slower but simpler' baseline; go straight to
# the stack-based evaluation, which is already optimal.
```

3. Complexity: N/A time, N/A space

4. Bottleneck: N/A for this one -- postfix is meant to be evaluated with a stack directly; there's no meaningfully slower correct alternative.

5. Optimize: Postfix means an operator always combines the two MOST RECENT operands — a stack naturally holds 'most recent' in order.

6. Code it (narrate as you write):

```
def eval_rpn(tokens):
    stack = []
    ops = {'+', '-', '*', '/'}
    for tok in tokens:
        if tok not in ops:
            stack.append(int(tok))
        else:
            b = stack.pop()
            a = stack.pop()
            if tok == '+': stack.append(a + b)
            elif tok == '-': stack.append(a - b)
            elif tok == '*': stack.append(a * b)
            else: stack.append(int(a / b)) # truncate toward 0
    return stack.pop()
```

9. Final complexity: O(n) time, O(n) space

7. Test: ["2","1","+","3","*"] -> push 2,1; '+' -> 3; push 3; '*' -> 9. Edge: negative division like -7/2 needs int(a/b), not a//b.

8. Other edge cases: single-number expression, no operators; division resulting in an exact integer; deeply nested operator chains.

Memory hook: "Numbers pile up; an operator pops the top two and pushes the result."

Implement Queue using Stacks

LeetCode #232 | **EASY** | Pattern: Stack (Two-Stack Trick) | Video: "Implement Queue using Stacks - Leetcode 232 - Python" — NeetCodeIO.
Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Implement a FIFO queue using only two stacks, supporting push, pop, peek, and empty. *Ask:* Are push/pop/peek/empty all that's required? Any constraint on amortized vs. worst-case time?

2. Brute force: Use one stack for input; to pop/peek (need the OLDEST item), pop everything into a temp list, take the item, then push everything back — done every single call.

```
class MyQueue:
    def __init__(self):
        self.stack = []
    def push(self, x):
        self.stack.append(x)
    def pop(self):
        temp = []
        while len(self.stack) > 1:
            temp.append(self.stack.pop())
        result = self.stack.pop()
        while temp:
            self.stack.append(temp.pop())
        return result # O(n) every call!
```

3. Complexity: pop/peek $O(n)$ every call time, $O(n)$ space

4. Bottleneck: Reversing the whole stack on every single pop/peek call redoes the same shuffling repeatedly.

5. Optimize: Two stacks: 'in_stack' for pushes, 'out_stack' for pops. Only transfer in->out when out_stack is empty; each element is moved at most once, ever.

6. Code it (narrate as you write):

```
class MyQueue:
    def __init__(self):
        self.in_stack = []
        self.out_stack = []
    def push(self, x):
        self.in_stack.append(x)
    def _shift(self):
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())
    def pop(self):
        self._shift()
        return self.out_stack.pop()
    def peek(self):
        self._shift()
        return self.out_stack[-1]
    def empty(self):
        return not self.in_stack and not self.out_stack
```

9. Final complexity: $O(1)$ amortized time, $O(n)$ space

7. Test: push(1), push(2), peek()->1, pop()->1, empty()->False. Edge: interleaved push/pop -- out_stack only refills once fully drained, preserving order.

8. Other edge cases: popping until truly empty; a single element; many pushes with no pops in between.

Memory hook: "Two stacks: reverse once into 'out', drain it fully before refilling."

Implement Stack using Queues

LeetCode #225 | **EASY** | Pattern: Queue (Rotation Trick) | Video: "Implement Stack using Queues - Leetcode 225 - Python" — NeetCode.
Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Implement a LIFO stack using only queues, supporting push, pop, top, and empty. Ask: Are push/pop/top/empty all required? Push allowed to be $O(n)$ if pop/top stay $O(1)$?

2. Brute force: Use two queues; to push, add to the empty one, then move everything from the other queue behind it, then swap names — $O(n)$ per push done via an extra queue.

```
from collections import deque

class MyStack:
    def __init__(self):
        self.q1 = deque()
        self.q2 = deque()
    def push(self, x):
        self.q2.append(x)
        while self.q1:
            self.q2.append(self.q1.popleft())
        self.q1, self.q2 = self.q2, self.q1
    def pop(self):
        return self.q1.popleft()
    def top(self):
        return self.q1[0]
    def empty(self):
        return not self.q1
```

3. Complexity: push $O(n)$ time, $O(n)$ space

4. Bottleneck: Using two full queues and swapping them every push does more bookkeeping than necessary.

5. Optimize: Single queue: after appending the new element, rotate the queue by rotating everything BEFORE it to behind it — the new element ends up at the front, acting like the top of a stack.

6. Code it (narrate as you write):

```
from collections import deque

class MyStack:
    def __init__(self):
        self.q = deque()
    def push(self, x):
        self.q.append(x)
        for _ in range(len(self.q) - 1):
            self.q.append(self.q.popleft())
    def pop(self):
        return self.q.popleft()
    def top(self):
        return self.q[0]
    def empty(self):
        return not self.q
```

9. Final complexity: push $O(n)$, pop/top $O(1)$ time, $O(n)$ space

7. Test: push(1), push(2), top() → 2, pop() → 2. Edge: single-element stack -- rotation loop runs zero times, element stays at front.

8. Other edge cases: popping to empty; pushing then immediately popping repeatedly; large sequential pushes.

Memory hook: "Rotate the queue so the newest push lands at the front."

Maximum Depth of Binary Tree

LeetCode #104 | **EASY** | Pattern: DFS / Recursion | Video: "Maximum Depth of Binary Tree - 3 Solutions - Leetcode 104 - Python" —

NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given the root of a binary tree, return its maximum depth (number of nodes on the longest root-to-leaf path). Ask: Depth counted in nodes (not edges)? Empty tree has depth 0?

2. Brute force: There isn't a distinct slower approach here; the natural solution IS the recursive one below. An alternative worth showing is iterative BFS (level counting).

```
from collections import deque

def max_depth_bfs(root):
    if not root:
        return 0
    depth = 0
    queue = deque([root])
    while queue:
        depth += 1
        for _ in range(len(queue)):
            node = queue.popleft()
            if node.left: queue.append(node.left)
            if node.right: queue.append(node.right)
    return depth
```

3. Complexity: $O(n)$ time, $O(n)$ worst case space

7. Test: root=3, left=9, right=20(15,7) -> left depth 1, right depth 2, total $1 + \max(1, 2) = 3$. Edge: empty tree (root=None) -> 0.

8. Other edge cases: single-node tree (depth 1); completely skewed tree (depth = node count); perfectly balanced tree.

Memory hook: "1 plus whichever child subtree goes deeper."

4. Bottleneck: N/A here -- the recursive definition IS the efficient solution; only alternative is iterative BFS, not a slower one.

5. Optimize: Depth of a tree = $1 + \max(\text{depth of left subtree, depth of right subtree})$; null node has depth 0. Direct recursive definition.

6. Code it (narrate as you write):

```
def max_depth(root):
    if root is None:
        return 0
    left = max_depth(root.left)
    right = max_depth(root.right)
    return 1 + max(left, right)
```

9. Final complexity: $O(n)$ time, $O(h)$ space

Binary Tree Level Order Traversal

LeetCode #102 | **MEDIUM** | Pattern: BFS | Video: "Binary Tree Level Order Traversal - BFS - Leetcode 102" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given the root of a binary tree, return the node values grouped level by level (left to right), as a list of lists. Ask: Return values grouped by level, left to right within each level? Empty tree returns []?

2. Brute force: DFS could work but requires tracking depth explicitly and appending to the right list — more error-prone than BFS's natural level grouping.

```
def level_order_dfs(root):
    result = []
    def dfs(node, depth):
        if not node:
            return
        if depth == len(result):
            result.append([])
        result[depth].append(node.val)
        dfs(node.left, depth + 1)
        dfs(node.right, depth + 1)
    dfs(root, 0)
    return result
```

3. Complexity: O(n) time, O(n) space

4. Bottleneck: DFS with explicit depth tracking works but is more error-prone than BFS's natural level grouping.

5. Optimize: BFS naturally processes one level at a time. Snapshot the queue's current size before processing, so that batch is exactly one level's worth of nodes.

6. Code it (narrate as you write):

```
from collections import deque

def level_order(root):
    if not root:
        return []
    result = []
    queue = deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left: queue.append(node.left)
            if node.right: queue.append(node.right)
        result.append(level)
    return result
```

9. Final complexity: O(n) time, O(n) space

7. Test: root=3,left=9,right=20(15,7) -> [[3],[9,20],[15,7]]. Edge: empty tree -> [] immediately.

8. Other edge cases: single-node tree; only left children (skewed); only right children (skewed).

Memory hook: "Snapshot queue size = process exactly one level."

Binary Tree Vertical Order Traversal

LeetCode #314 | **MEDIUM** | Pattern: BFS + Hash Map (Column Index) | Video: "Binary Tree Vertical Order Traversal - Leetcode 314 - Python" — NeetCodeIO. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given a binary tree, return the node values grouped by vertical column, ordered left to right; within a column, top to bottom, and left-to-right for ties at the same node depth. *Ask:* Ties at the same column AND same depth ordered left to right? Column indices can be negative?

2. Brute force: DFS while tracking (column, row) pairs, then sort all (column, row, value) triples at the end.

```
def vertical_order_dfs(root):
    triples = []
    def dfs(node, col, row):
        if not node:
            return
        triples.append((col, row, node.val))
        dfs(node.left, col - 1, row + 1)
        dfs(node.right, col + 1, row + 1)
    dfs(root, 0, 0)
    triples.sort() # by col, then row, then value
    result = {}
    for col, row, val in triples:
        result.setdefault(col, []).append(val)
    return [result[c] for c in sorted(result)]
```

3. Complexity: $O(n \log n)$ time, $O(n)$ space

4. Bottleneck: A DFS-then-sort-everything approach does more sorting than needed when BFS already visits top-to-bottom, left-to-right.

5. Optimize: BFS processes nodes level by level (top to bottom, left to right) automatically, so within a column, values are already appended in correct order — no final sort needed, just group by column index and sort column KEYS at the end.

6. Code it (narrate as you write):

```
from collections import deque, defaultdict

def vertical_order(root):
    if not root:
        return []
    columns = defaultdict(list)
    queue = deque([(root, 0)])
    while queue:
        node, col = queue.popleft()
        columns[col].append(node.val)
        if node.left: queue.append((node.left, col - 1))
        if node.right: queue.append((node.right, col + 1))
    return [columns[c] for c in sorted(columns)]
```

9. Final complexity: $O(n \log n)$ time, $O(n)$ space

7. Test: root=3, left=9, right=8 (left=4) -> columns -1:[9,4], 0:[3,8]? trace carefully per structure. Edge: two nodes same column+level from different parents -- BFS order still resolves left-to-right correctly.

8. Other edge cases: empty tree (return []); single node (one column); all nodes in one column (a straight line).

Memory hook: "BFS order is already top-to-bottom, left-to-right — just bucket by column."

Clone Graph

LeetCode #133 | **MEDIUM** | Pattern: DFS/BFS + Hash Map | Video: "Clone Graph - Depth First Search - Leetcode 133" — NeetCode.

Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given a reference node in a connected undirected graph, return a deep copy (clone) of the entire graph. Ask: Graph is connected and undirected? Guaranteed no self-loops beyond what's explicit in neighbors lists?

2. Brute force: There isn't a meaningfully 'slower' correct alternative — you fundamentally need some way to avoid re-cloning (and infinitely looping on) a node you've already visited. Naive recursion WITHOUT a visited map would infinite-loop on any cycle.

```
# Recursion without tracking visited nodes:
def clone_graph_broken(node):
    if not node:
        return None
    copy = Node(node.val)
    for neighbor in node.neighbors:
        copy.neighbors.append(clone_graph_broken(neighbor))
    return copy
# BROKEN: infinite recursion on any cycle back to a
# node already being cloned (graphs here are typically cycl
```

3. Complexity: Infinite loop on cycles time, N/A space

4. Bottleneck: Recursing without tracking visited nodes infinite-loops on any cycle, which this graph type has.

5. Optimize: A hash map from original node to its clone both (a) prevents infinite recursion on cycles and (b) lets neighbors correctly reference already-created clones.

6. Code it (narrate as you write):

```
def clone_graph(node):
    if not node:
        return None
    old_to_new = {}
    def dfs(n):
        if n in old_to_new:
            return old_to_new[n]
        copy = Node(n.val)
        old_to_new[n] = copy
        for neighbor in n.neighbors:
            copy.neighbors.append(dfs(neighbor))
        return copy
    return dfs(node)
```

9. Final complexity: $O(V + E)$ time, $O(V)$ space

7. Test: node 1 with neighbors [2,4] -> clone recurses, map prevents re-cloning shared neighbors. Edge: single node with itself as a neighbor -- map entry created before recursing avoids infinite loop.

8. Other edge cases: single node with no neighbors; a simple cycle of 3 nodes; disconnected part (not expected, but worth asking).

Memory hook: "Map original -> clone FIRST, then recurse into neighbors."

Course Schedule

LeetCode #207 | **MEDIUM** | Pattern: Graph DFS / Topological Sort (Cycle Detection) | Video: "Course Schedule - Graph Adjacency List - Leetcode 207" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given numCourses and prerequisite pairs [a, b] (must take b before a), return whether it's possible to finish all courses (i.e., the prerequisite graph has no cycle). Ask: Prerequisites given as [a,b] meaning b before a? Can there be duplicate prerequisite pairs?

2. Brute force: Try every possible course order (permutations) and check if any order satisfies all prerequisites — combinatorially expensive.

```
from itertools import permutations

def can_finish_brute(numCourses, prerequisites):
    for order in permutations(range(numCourses)):
        position = {c: i for i, c in enumerate(order)}
        if all(position[b] < position[a] for a, b in prerequisites):
            return True
    return False # O(n!) - only feasible for tiny n
```

3. Complexity: $O(n!)$ time, $O(n)$ space

4. Bottleneck: Trying every possible course ordering directly is combinatorially expensive (factorial).

5. Optimize: A valid course order exists iff the prerequisite graph has no cycle. DFS each course, marking nodes 'in current path'; if DFS revisits a node still in the current path, that's a cycle.

6. Code it (narrate as you write):

```
def can_finish(numCourses, prerequisites):
    graph = [[] for _ in range(numCourses)]
    for a, b in prerequisites:
        graph[a].append(b)
    state = [0] * numCourses # 0=unvisited, 1=visiting, 2=done
    def dfs(course):
        if state[course] == 1:
            return False # cycle!
        if state[course] == 2:
            return True
        state[course] = 1
        for pre in graph[course]:
            if not dfs(pre):
                return False
        state[course] = 2
        return True
    return all(dfs(c) for c in range(numCourses))
```

9. Final complexity: $O(V + E)$ time, $O(V + E)$ space

7. Test: numCourses=2, prereqs=[[1,0]] -> course 0 has no deps, course 1 depends on 0, no cycle, True. Edge: [[0,0]] -- course depends on itself, DFS revisits itself mid-path, cycle detected, False.

8. Other edge cases: no prerequisites at all (always True); a long chain with no cycle; disconnected groups of courses.

Memory hook: "Mark 'in-progress'; revisiting an in-progress node means a cycle."

Binary Search

LeetCode #704 | **EASY** | Pattern: Binary Search | Video: "Binary Search - Leetcode 704 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given a sorted array of integers and a target, return its index, or -1 if not present, in $O(\log n)$. Ask: Array guaranteed sorted ascending? Duplicate values possible (does it matter which index is returned)?

2. Brute force: Scan every element left to right until the target is found.

```
def search(nums, target):
    for i, num in enumerate(nums):
        if num == target:
            return i
    return -1
```

3. Complexity: $O(n)$ time, $O(1)$ space

4. Bottleneck: Scanning left to right ignores the sorted order that lets me eliminate half the array per comparison.

5. Optimize: Sorted order means comparing the middle element tells you which entire half to discard — halving the search space every step.

6. Code it (narrate as you write):

```
def search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

9. Final complexity: $O(\log n)$ time, $O(1)$ space

7. Test: [-1,0,3,5,9,12], target=9 -> mid narrows down, found at index 4. Edge: target not present, e.g. target=2 -> left crosses right, return -1.

8. Other edge cases: single-element array; target is the first or last element; empty array.

Memory hook: "Middle tells you which whole half to throw away."

Find Minimum in Rotated Sorted Array

LeetCode #153 | **MEDIUM** | Pattern: Binary Search (Modified) | Video: "Find Minimum in Rotated Sorted Array - Binary Search - Leetcode 153 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given a rotated sorted array with all-unique values, find the minimum element in $O(\log n)$. Ask: All values unique guaranteed? Could the array be not rotated at all (already sorted)?

2. Brute force: Scan the whole array to find the minimum directly.

```
def find_min(nums):  
    return min(nums) # O(n), ignores the sorted structure
```

3. Complexity: $O(n)$ time, $O(1)$ space

4. Bottleneck: Scanning every element to find the min ignores the sorted-then-rotated structure that binary search can exploit.

5. Optimize: Compare $\text{nums}[\text{mid}]$ to $\text{nums}[\text{right}]$. If $\text{nums}[\text{mid}] > \text{nums}[\text{right}]$, the minimum is in the right half (rotation point is there); otherwise it's in the left half (mid could BE the minimum, so don't exclude it).

6. Code it (narrate as you write):

```
def find_min(nums):  
    left, right = 0, len(nums) - 1  
    while left < right:  
        mid = (left + right) // 2  
        if nums[mid] > nums[right]:  
            left = mid + 1  
        else:  
            right = mid  
    return nums[left]
```

9. Final complexity: $O(\log n)$ time, $O(1)$ space

7. Test: [3,4,5,1,2] -> $\text{mid}=5 > \text{right}(2)$ so search right half; converges to index 3, value 1. Edge: not rotated, e.g. [1,2,3,4,5] -> $\text{nums}[\text{mid}]$ never exceeds $\text{nums}[\text{right}]$, converges to index 0.

8. Other edge cases: rotation point at the very start (no rotation); rotation point at the very end; two-element array.

Memory hook: "Compare mid to the right edge — that tells you which side hides the rotation."

LRU Cache

LeetCode #146 | **MEDIUM** | Pattern: Hash Map + Doubly Linked List | Video: "LRU Cache - Twitch Interview Question - Leetcode 146" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Design a Least Recently Used (LRU) cache with a fixed capacity, supporting `get(key)` and `put(key, value)`, both in $O(1)$ average time. Ask: Capacity guaranteed at least 1? Does 'use' include both `get` AND `put` (both refresh recency)?

2. Brute force: Use a plain dict for values plus a separate list tracking usage order; every `get/put` requires an $O(n)$ scan or shift of that order list.

```
class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.data = {}
        self.order = [] # least-recent at index 0
    def get(self, key):
        if key not in self.data:
            return -1
        self.order.remove(key) # O(n)
        self.order.append(key)
        return self.data[key]
    def put(self, key, value):
        if key in self.data:
            self.order.remove(key) # O(n)
        elif len(self.data) >= self.cap:
            oldest = self.order.pop(0) # O(n)
            del self.data[oldest]
        self.data[key] = value
        self.order.append(key)
```

3. Complexity: $O(n)$ per `get/put` time, $O(n)$ space

4. Bottleneck: Using a plain list to track recency order means removing/reordering an item costs $O(n)$ (shifting).

5. Optimize: A hash map gives $O(1)$ lookup by key; a doubly linked list gives $O(1)$ removal/insertion at any position (no shifting) to track recency order, moving a node to the front on every access.

6. Code it (narrate as you write):

```
class Node:
    def __init__(self, k, v):
        self.key, self.val = k, v
        self.prev = self.next = None

class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.map = {}
        self.left = Node(0, 0) # least recent side
        self.right = Node(0, 0) # most recent side
        self.left.next = self.right
        self.right.prev = self.left
    def _remove(self, node):
        node.prev.next, node.next.prev = node.next, node.prev
    def _insert(self, node):
        node.prev, node.next = self.right.prev, self.right
        self.right.prev.next = node
        self.right.prev = node
    def get(self, key):
        if key not in self.map:
            return -1
        self._remove(self.map[key]); self._insert(self.map[key])
        return self.map[key].val
    def put(self, key, value):
        if key in self.map:
            self._remove(self.map[key])
        node = Node(key, value)
        self.map[key] = node
        self._insert(node)
        if len(self.map) > self.cap:
            lru = self.left.next
            self._remove(lru); del self.map[lru.key]
```

9. Final complexity: $O(1)$ `get/put` time, $O(\text{capacity})$ space

7. Test: capacity=2: `put(1,1), put(2,2), get(1)->1, put(3,3)` evicts key 2, `get(2)->-1`. Edge: updating an existing key's value must move it to most-recent, not just overwrite in place.

8. Other edge cases: capacity of 1 (every `put` evicts the previous key); getting a key right after inserting it; repeated `puts` to the same key.

Memory hook: "Map for $O(1)$ lookup, doubly linked list for $O(1)$ reorder — evict from the far end."

Random Pick with Weight

LeetCode #528 | **MEDIUM** | Pattern: Prefix Sum + Binary Search | Video: "Random Pick with Weight | Bucket approach | Leetcode #528 | Binary search" — Techdose. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given an array of positive weights, design a structure that picks an index at random, where the probability of picking index i is proportional to $\text{weights}[i]$. Ask: Weights guaranteed positive integers? Does `pick_index` get called many times (yes, so precompute once)?

2. Brute force: Build a 'bucket' array where each index appears $\text{weights}[i]$ times, then pick a uniformly random element from that array directly.

```
import random

class Solution:
    def __init__(self, w):
        self.buckets = []
        for i, weight in enumerate(w):
            self.buckets.extend([i] * weight) # O(total weights)
    def pick_index(self):
        return random.choice(self.buckets)
```

3. Complexity: build $O(\sum w)$, pick $O(1)$ time, $O(\sum w)$ space

4. Bottleneck: Building a bucket array of size $\sum(\text{weights})$ wastes huge memory when weights are large.

5. Optimize: Build a prefix-sum array of weights (cumulative ranges). Pick a random number in $[1, \text{total}]$, then binary search for the first prefix sum $\geq$ that number — that index's range contains the random point.

6. Code it (narrate as you write):

```
import random, bisect

class Solution:
    def __init__(self, w):
        self.prefix = []
        total = 0
        for weight in w:
            total += weight
            self.prefix.append(total)
        self.total = total
    def pick_index(self):
        target = random.randint(1, self.total)
        return bisect.bisect_left(self.prefix, target)
```

9. Final complexity: build $O(n)$, pick $O(\log n)$ time, $O(n)$ space

7. Test: $w=[1,3] \rightarrow \text{prefix}=[1,4]$, $\text{total}=4$; random in $[1,4]$: 1 $\rightarrow$ index 0, 2..4 $\rightarrow$ index 1 (3x more likely, matching weight). Edge: single weight of 1 $\rightarrow$ always index 0.

8. Other edge cases: all weights equal (uniform pick); one very large weight dominating; two weights only.

Memory hook: "Weights become ranges (prefix sums); binary search finds which range the dart landed in."

Rotting Oranges

LeetCode #994 | **MEDIUM** | Pattern: Multi-Source BFS | Video: "Rotting Oranges - Leetcode 994 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: A grid has empty cells, fresh oranges, and rotten oranges. Every minute, rot spreads to adjacent fresh oranges. Return the minutes until no fresh orange remains, or -1 if impossible. Ask: 4-directional spread only, not diagonal? Return 0 if no fresh oranges exist at the start?

2. Brute force: Simulate minute by minute: rescan the WHOLE grid every minute to find newly-infected oranges.

```
def oranges_rotting_brute(grid):
    rows, cols = len(grid), len(grid[0])
    minutes = 0
    while True:
        fresh_left = False
        to_rot = []
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1:
                    fresh_left = True
                    for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
                        nr, nc = r+dr, c+dc
                        if 0<=nr<rows and 0<=nc<cols and grid[nr][nc] == 0:
                            to_rot.append((nr,nc))
                            break
        if not to_rot:
            return -1 if fresh_left else minutes
        for r, c in to_rot:
            grid[r][c] = 2
        minutes += 1
```

3. Complexity: $O((rows*cols)^2)$ time, $O(rows*cols)$ space

4. Bottleneck: Rescanning the entire grid every single minute to find newly-infected oranges redoes work already known.

5. Optimize: Seed a BFS queue with EVERY rotten orange at once. Each full BFS level = exactly one minute of simultaneous spreading; no rescanning needed.

6. Code it (narrate as you write):

```
from collections import deque

def oranges_rotting(grid):
    rows, cols = len(grid), len(grid[0])
    queue = deque()
    fresh = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2: queue.append((r, c))
            elif grid[r][c] == 1: fresh += 1
    minutes = 0
    dirs = [(-1,0), (1,0), (0,-1), (0,1)]
    while queue and fresh > 0:
        minutes += 1
        for _ in range(len(queue)):
            r, c = queue.popleft()
            for dr, dc in dirs:
                nr, nc = r+dr, c+dc
                if 0<=nr<rows and 0<=nc<cols and grid[nr][nc]==1:
                    grid[nr][nc] = 2
                    fresh -= 1
                    queue.append((nr, nc))
    return minutes if fresh == 0 else -1
```

9. Final complexity: $O(rows * cols)$ time, $O(rows * cols)$ space

7. Test: $[[2,1,1],[1,1,0],[0,1,1]] \rightarrow$ rot spreads outward minute by minute, takes 4 minutes total. Edge: a fresh orange sealed off by walls with no path to any rotten orange $\rightarrow$ fresh never hits 0, return -1.

8. Other edge cases: no fresh oranges at all (return 0); no rotten oranges but fresh ones exist (return -1); grid that's entirely empty cells.

Memory hook: "Seed BFS with every rotten orange at once — BFS levels ARE the minutes."

Maximum Subarray

LeetCode #53 | **MEDIUM** | Pattern: Dynamic Programming (Kadane's Algorithm) | Video: "Maximum Subarray - Amazon Coding Interview Question - Leetcode 53 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given an integer array (possibly with negatives), find the contiguous subarray with the largest sum and return that sum. Ask: Array guaranteed non-empty? Must return a real subarray sum (can't return 0 for an empty selection)?

2. Brute force: Check every possible subarray's sum directly (or extend a running sum per start index).

```
def max_subarray(nums):
    best = nums[0]
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            best = max(best, total)
    return best
```

3. Complexity: $O(n^2)$ time, $O(1)$ space

4. Bottleneck: Recomputing every subarray's sum from scratch redoes overlapping additions across nearby subarrays.

5. Optimize: If the running sum ever goes negative, it can only hurt any future subarray — drop it and restart at the current element instead of carrying it forward.

6. Code it (narrate as you write):

```
def max_subarray(nums):
    current = best = nums[0]
    for num in nums[1:]:
        current = max(num, current + num)
        best = max(best, current)
    return best
```

9. Final complexity: $O(n)$ time, $O(1)$ space

7. Test: [-2,1,-3,4,-1,2,1,-5,4] -> running sum resets after negatives, builds up to best=6 via [4,-1,2,1]. Edge: all-negative array, e.g. [-3,-1,-2] -- starting at nums[0] (not 0) correctly gives -1.

8. Other edge cases: single-element array; entire array is the answer (all positive); array with a single large positive surrounded by negatives.

Memory hook: "A negative running sum is dead weight — drop it and start fresh."

Top K Frequent Elements

LeetCode #347 | **MEDIUM** | Pattern: Bucket Sort (or Heap) | Video: "Top K Frequent Elements - Bucket Sort - Leetcode 347 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given an integer array and an integer k, return the k most frequently occurring elements, in any order. Ask: Order of the output doesn't matter? k guaranteed $\leq$ number of distinct elements?

2. Brute force: Count frequencies, then fully sort all distinct values by count and take the top k.

```
from collections import Counter

def top_k_frequent(nums, k):
    counts = Counter(nums)
    sorted_items = sorted(counts.items(), key=lambda x: -x[1])
    return [val for val, cnt in sorted_items[:k]]
```

3. Complexity: $O(n \log n)$ time, $O(n)$ space

4. Bottleneck: Fully sorting all distinct values by frequency does more ordering work than needed when I only want the top k.

5. Optimize: Frequency can never exceed n, so make 'buckets' indexed by frequency (0..n); put each value in its frequency's bucket, then read off values from the highest bucket down until k are collected — no comparison sort needed.

6. Code it (narrate as you write):

```
from collections import Counter

def top_k_frequent(nums, k):
    counts = Counter(nums)
    n = len(nums)
    buckets = [[] for _ in range(n + 1)]
    for val, freq in counts.items():
        buckets[freq].append(val)
    result = []
    for freq in range(n, 0, -1):
        for val in buckets[freq]:
            result.append(val)
            if len(result) == k:
                return result
    return result
```

9. Final complexity: $O(n)$ time, $O(n)$ space

7. Test: [1,1,1,2,2,3], k=2 -> counts {1:3,2:2,3:1}; buckets collect from freq=3 down, stop once 2 collected -> [1,2]. Edge: k equals number of distinct elements -> loop collects everything before returning.

8. Other edge cases: all elements with the same frequency; k=1 (single most frequent); many low-frequency singletons.

Memory hook: "Bucket index = frequency — no sorting needed, frequency is bounded by n."

Maximum Product Subarray

LeetCode #152 | **MEDIUM** | Pattern: DP (Track Max & Min) | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given an integer array, find the contiguous subarray with the largest product and return that product. Ask: Can the array contain zero(s)? Negative numbers guaranteed possible?

2. Brute force: Check every subarray's product directly.

```
def max_product(nums):
    best = nums[0]
    for i in range(len(nums)):
        product = 1
        for j in range(i, len(nums)):
            product *= nums[j]
            best = max(best, product)
    return best
```

3. Complexity: $O(n^2)$ time, $O(1)$ space

4. Bottleneck: A single running max can't handle negatives -- a very negative running product times another negative can become the new best, so max alone loses information.

5. Optimize: Track both current_max and current_min ending at each index. On a negative number, swap them before combining, since multiplying by a negative flips which one becomes bigger.

6. Code it (narrate as you write):

```
def max_product(nums):
    result = cur_max = cur_min = nums[0]
    for num in nums[1:]:
        if num < 0:
            cur_max, cur_min = cur_min, cur_max
        cur_max = max(num, cur_max * num)
        cur_min = min(num, cur_min * num)
        result = max(result, cur_max)
    return result
```

9. Final complexity: $O(n)$ time, $O(1)$ space

7. Test: [2,3,-2,4] -> max builds to 6 via [2,3], dips at -2, 4 alone can't beat 6, result stays 6. Edge: zero in the array, e.g. [-2,0,-1] -> zero resets both max/min to 0, result=0.

8. Other edge cases: single negative number; even count of negatives (whole array is the answer); all zeros.

Memory hook: "Two negatives make a positive -- track the min too, not just the max."

Find the Duplicate Number

LeetCode #287 | **MEDIUM** | Pattern: Fast & Slow Pointers | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given an array of $n+1$ integers with values in $[1, n]$, exactly one value repeats (possibly more than twice); find it without modifying the array, in $O(1)$ space. Ask: Can the duplicate value repeat more than twice? Must the array stay unmodified?

2. Brute force: Hash set: return the first value already seen.

```
def find_duplicate(nums):
    seen = set()
    for num in nums:
        if num in seen:
            return num
    seen.add(num)
```

3. Complexity: $O(n)$ time, $O(n)$ space

4. Bottleneck: The hash set uses $O(n)$ memory just to remember visited values, when the values themselves (being valid indices) can act as pointers with no extra memory.

5. Optimize: Index i 'points to' index $nums[i]$. A duplicate means two indices point to the same place -- a cycle. Floyd's Tortoise & Hare finds the cycle's entrance, which is the duplicate.

6. Code it (narrate as you write):

```
def find_duplicate(nums):
    slow = fast = nums[0]
    while True:
        slow = nums[slow]
        fast = nums[nums[fast]]
        if slow == fast:
            break
    slow2 = nums[0]
    while slow2 != slow:
        slow2 = nums[slow2]
        slow = nums[slow]
    return slow
```

9. Final complexity: $O(n)$ time, $O(1)$ space

7. Test: $[1, 3, 4, 2, 2]$ -> phase 1 meets at 2, phase 2 (reset+walk together) also converges at 2. Edge: duplicate repeats 3+ times, e.g. $[1, 2, 2, 2]$ with $n=3$ -> cycle structure still holds, still converges correctly.

8. Other edge cases: duplicate equal to n ; duplicate equal to 1; smallest array $n=1$ -> $[1, 1]$.

Memory hook: "Values point to indices -- a duplicate makes a loop, hunt it like a cycle."

Single Number

LeetCode #136 | **EASY** | Pattern: Bit Manipulation (XOR) | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Every element in a non-empty array appears exactly twice except one; find that single element in $O(n)$ time, $O(1)$ space. *Ask:* Guaranteed exactly one element appears once, all others exactly twice (not three+)?

2. Brute force: Hash map counts, then scan for the count-1 entry.

```
from collections import Counter

def single_number(nums):
    counts = Counter(nums)
    for val, cnt in counts.items():
        if cnt == 1:
            return val
```

3. Complexity: $O(n)$ time, $O(n)$ space

4. Bottleneck: Storing every value's count uses $O(n)$ memory just to detect pairs, when an operation exists that cancels pairs automatically with no memory.

5. Optimize: $x \text{ XOR } x = 0$ and $x \text{ XOR } 0 = x$. XOR-ing the whole array cancels every duplicate pair, leaving only the singleton.

6. Code it (narrate as you write):

```
def single_number(nums):
    result = 0
    for num in nums:
        result ^= num
    return result
```

9. Final complexity: $O(n)$ time, $O(1)$ space

7. Test: [4,1,2,1,2] -> XOR-ing all: the two 1's cancel, the two 2's cancel, leaves 4. Edge: single-element array [7] -> $0 \text{ XOR } 7 = 7$.

8. Other edge cases: negative numbers (XOR works the same on two's complement); the singleton is 0 itself; singleton at the start vs end.

Memory hook: "XOR cancels pairs -- whatever survives is the one without a partner."

Move Zeroes

LeetCode #283 | **EASY** | Pattern: Two Pointers (In-Place Partition) | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given an array, move all zeroes to the end while keeping the relative order of non-zero elements, in-place. Ask: Must this be in-place (no new array)? Keep non-zero elements' original relative order?

2. Brute force: Build a new list of non-zeros, then zeroes, and copy back over the original.

```
def move_zeroes(nums):
    non_zero = [x for x in nums if x != 0]
    zero_count = len(nums) - len(non_zero)
    nums[:] = non_zero + [0] * zero_count
```

3. Complexity: $O(n)$ time, $O(n)$ space

4. Bottleneck: Building a second array uses $O(n)$ extra space when the same array can be rearranged directly with two pointers.

5. Optimize: insert_pos tracks where the next non-zero should go. Scan with i; on a non-zero, swap it into insert_pos and advance -- the swap pushes the displaced zero further right automatically.

6. Code it (narrate as you write):

```
def move_zeroes(nums):
    insert_pos = 0
    for i in range(len(nums)):
        if nums[i] != 0:
            nums[insert_pos], nums[i] = nums[i], nums[insert_pos]
            insert_pos += 1
```

9. Final complexity: $O(n)$ time, $O(1)$ space

7. Test: [0,1,0,3,12] -> swaps build up to [1,3,12,0,0]. Edge: no zeroes at all, e.g. [1,2,3] -> every swap is with itself, array unchanged.

8. Other edge cases: all zeroes; single element; zeroes already all at the end.

Memory hook: "Slow pointer marks the next non-zero slot; swap it in as you scan."

Two Sum II - Input Array Is Sorted

LeetCode #167 | **MEDIUM** | Pattern: Two Pointers | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given a sorted array and a target, find two numbers that add up to it and return their 1-indexed positions. Ask: Array guaranteed sorted ascending? Exactly one valid pair? 1-indexed output?

2. Brute force: Check every pair of indices.

```
def two_sum(numbers, target):
    for i in range(len(numbers)):
        for j in range(i + 1, len(numbers)):
            if numbers[i] + numbers[j] == target:
                return [i + 1, j + 1]
```

3. Complexity: $O(n^2)$ time, $O(1)$ space

4. Bottleneck: Checking every pair ignores the sorted order, which tells you exactly which direction fixes an incorrect sum.

5. Optimize: Sorted order means moving the left pointer right can only increase the sum, moving right left can only decrease it -- narrow from both ends.

6. Code it (narrate as you write):

```
def two_sum(numbers, target):
    left, right = 0, len(numbers) - 1
    while left < right:
        total = numbers[left] + numbers[right]
        if total == target:
            return [left + 1, right + 1]
        elif total < target:
            left += 1
        else:
            right -= 1
```

9. Final complexity: $O(n)$ time, $O(1)$ space

7. Test: [2,7,11,15], target=9 -> 2+15 too big, shrink right twice, 2+7=9 -> [1,2]. Edge: matching pair are duplicate values -- pointers still converge correctly.

8. Other edge cases: smallest array (2 elements); negative numbers present; target needs first+last elements.

Memory hook: "Sorted array, so walk the pointers toward the target from both ends."

Boats to Save People

LeetCode #881 | **MEDIUM** | Pattern: Two Pointers + Greedy | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Each boat holds at most 2 people and has a weight limit. Given people's weights, find the minimum number of boats to carry everyone. Ask: Every person individually fits the limit alone? Boat can carry just 1 if pairing isn't possible?

2. Brute force: Try many pairings and take the best (no efficient structure).

```
from itertools import permutations

def num_rescue_boats_brute(people, limit):
    # illustrative only -- trying arrangements directly is
    # combinatorially expensive and not a practical baseline
    return None # see optimal; brute force here isn't meant
```

3. Complexity: Exponential (impractical) time, $O(n)$ space

4. Bottleneck: Without sorting, there's no efficient way to know which two people are the best pairing candidates.

5. Optimize: Sort, then two pointers: try pairing the lightest with the heaviest remaining. If they fit, both board one boat; if not, the heaviest goes alone -- it can't fit with anyone lighter either.

6. Code it (narrate as you write):

```
def num_rescue_boats(people, limit):
    people.sort()
    left, right = 0, len(people) - 1
    boats = 0
    while left <= right:
        if people[left] + people[right] <= limit:
            left += 1
            right -= 1
        else:
            boats += 1
            right -= 1
    return boats
```

9. Final complexity: $O(n \log n)$ time, $O(1)$ extra space

7. Test: [3,2,2,1], limit=3 -> sorted [1,2,2,3]: 1+3>3 (3 alone), 1+2<=3 (pair), 2 alone -> 3 boats. Edge: everyone weighs exactly the limit -> nobody ever pairs, n boats.

8. Other edge cases: single person; all pairs fit perfectly (even n); odd number of people with one left over.

Memory hook: "Pair the heaviest with the lightest -- if that fails, no one else fits either."

Minimum Size Subarray Sum

LeetCode #209 | **MEDIUM** | Pattern: Sliding Window (Variable Size) | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given positive integers and a target, find the minimal length of a contiguous subarray whose sum is $\geq$ target, or 0 if none exists. Ask: All numbers guaranteed positive? Return 0 if no valid subarray exists?

2. Brute force: Check every subarray's sum directly.

```
def min_subarray_len(target, nums):
    best = float('inf')
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            if total >= target:
                best = min(best, j - i + 1)
                break
    return 0 if best == float('inf') else best
```

3. Complexity: $O(n^2)$ time, $O(1)$ space

4. Bottleneck: Recomputing sums for overlapping subarrays redoes additions already done; since all numbers are positive, a window can grow/shrink incrementally instead.

5. Optimize: Grow the window right, adding to a running sum. Whenever sum $\geq$ target, shrink from the left as far as possible, recording the smallest valid length.

6. Code it (narrate as you write):

```
def min_subarray_len(target, nums):
    left = 0
    total = 0
    best = float('inf')
    for right in range(len(nums)):
        total += nums[right]
        while total >= target:
            best = min(best, right - left + 1)
            total -= nums[left]
            left += 1
    return 0 if best == float('inf') else best
```

9. Final complexity: $O(n)$ time, $O(1)$ space

7. Test: target=7, [2,3,1,2,4,3] -> window shrinks/grows, smallest valid window is [4,3], length 2. Edge: target unreachable, e.g. target=100 on small numbers -> stays inf, return 0.

8. Other edge cases: whole array needed to reach target; single element already $\geq$ target; array of length 1.

Memory hook: "Grow until it's enough, then shrink until it's not."

Find All Anagrams in a String

LeetCode #438 | **MEDIUM** | Pattern: Sliding Window (Fixed Size) | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given strings *s* and *p*, find all starting indices in *s* where a substring is an anagram of *p*. Ask: Lowercase English letters only? Overlapping matches should all count?

2. Brute force: For every start index, sort a same-length window and compare to sorted *p*.

```
def find_anagrams(s, p):
    result = []
    k = len(p)
    target = sorted(p)
    for i in range(len(s) - k + 1):
        if sorted(s[i:i+k]) == target:
            result.append(i)
    return result
```

3. Complexity: $O(n * k \log k)$ time, $O(k)$ space

4. Bottleneck: Re-sorting a whole new window at every position redoes work that overlaps almost entirely with the previous window.

5. Optimize: Track a 26-length running count for the window; add the incoming char, remove the outgoing char, compare to *p*'s count array in $O(1)$ -ish per slide.

6. Code it (narrate as you write):

```
def find_anagrams(s, p):
    if len(p) > len(s):
        return []
    need = [0]*26
    window = [0]*26
    for ch in p:
        need[ord(ch)-97] += 1
    result = []
    for i in range(len(s)):
        window[ord(s[i])-97] += 1
        if i >= len(p):
            window[ord(s[i-len(p)])-97] -= 1
        if i >= len(p)-1 and window == need:
            result.append(i - len(p) + 1)
    return result
```

9. Final complexity: $O(n)$ time, $O(1)$ (26 slots) space

7. Test: *s*='cbaebabacd', *p*='abc' -> matches at indices 0 and 6. Edge: *p* longer than *s* -> early return [].

8. Other edge cases: *s* equals *p*; no anagram exists anywhere; *p* has repeated letters.

Memory hook: "Same letter counts, sliding window, add one / remove one."

Next Greater Element I

LeetCode #496 | **EASY** | Pattern: Monotonic Stack | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: nums1 is a subset of nums2; for each value in nums1, find its next greater element to the right in nums2, or -1. Ask: Every nums1 value guaranteed to exist in nums2? Duplicates possible in nums2?

2. Brute force: For each query value, find it in nums2 and scan forward for something bigger.

```
def next_greater_element(nums1, nums2):
    result = []
    for x in nums1:
        idx = nums2.index(x)
        found = -1
        for j in range(idx + 1, len(nums2)):
            if nums2[j] > x:
                found = nums2[j]
                break
        result.append(found)
    return result
```

3. Complexity: $O(n * m)$ time, $O(1)$ extra space

4. Bottleneck: Re-scanning nums2 for every query redoes work; the 'next greater' for every value in nums2 can be precomputed once.

5. Optimize: One pass over nums2 with a decreasing stack: when a bigger number appears, it resolves every smaller value still waiting on the stack.

6. Code it (narrate as you write):

```
def next_greater_element(nums1, nums2):
    next_greater = {}
    stack = []
    for num in nums2:
        while stack and stack[-1] < num:
            next_greater[stack.pop()] = num
        stack.append(num)
    return [next_greater.get(x, -1) for x in nums1]
```

9. Final complexity: $O(n + m)$ time, $O(n)$ space

7. Test: nums2=[2,1,2,4,3] -> map builds {1:2, 2:4}. nums1=[2,4] -> [4,-1] (4 never resolves). Edge: largest value at the very end -> never popped, defaults to -1.

8. Other edge cases: strictly decreasing nums2 (all -1); nums1 equals nums2; single-element nums2.

Memory hook: "Stack holds the unresolved; a bigger number resolves everyone smaller beneath it."

Decode String

LeetCode #394 | **MEDIUM** | Pattern: Stack | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Decode a string with pattern $k[\text{encoded}]$, meaning encoded repeats k times, with possible nesting, e.g. `'3[a2[c]]'`. Ask: Can nesting go arbitrarily deep? Are repeat counts always positive, possibly multi-digit?

2. Brute force: Repeatedly expand the innermost $k[\dots]$ with string replace until no brackets remain.

```
import re

def decode_string_brute(s):
    while '[' in s:
        s = re.sub(r'(\d+)\s*([a-z]*)\]',
                    lambda m: m.group(2) * int(m.group(1)),
                    s)
    return s # needs innermost-first matching; fragile for
```

3. Complexity: $O(n^2)$ or worse time, $O(n)$ space

4. Bottleneck: Repeated whole-string expansion rebuilds and rescans the string at every nesting level instead of handling nesting directly in one pass.

5. Optimize: A stack pauses the current (string, count) when '[' is seen, and resumes/combines it when ']' closes that level.

6. Code it (narrate as you write):

```
def decode_string(s):
    stack = []
    current_str = ''
    current_num = 0
    for ch in s:
        if ch.isdigit():
            current_num = current_num * 10 + int(ch)
        elif ch == '[':
            stack.append((current_str, current_num))
            current_str, current_num = '', 0
        elif ch == ']':
            prev_str, num = stack.pop()
            current_str = prev_str + current_str * num
        else:
            current_str += ch
    return current_str
```

9. Final complexity: $O(n * \text{maxK})$ time, $O(n)$ space

7. Test: `'3[a2[c]]'` -> inner resolves to 'acc', outer to 'accaccacc'. Edge: no brackets at all, e.g. 'abc' -> passes through unchanged.

8. Other edge cases: multiple sibling groups, e.g. `'2[a]3[b]'`; multi-digit counts like `'213[a]'`; deeply nested brackets.

Memory hook: "Bracket opens: pause and push. Bracket closes: pop and combine."

Simplify Path

LeetCode #71 | **MEDIUM** | Pattern: Stack | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given a Unix-style absolute path, simplify it into canonical form, resolving '.', '..', and collapsing repeated slashes. Ask: What happens if '.' would go above the root? Trailing slashes need removing?

2. Brute force: Directly manipulate the raw string, searching backward for the previous slash on every '..'.

```
def simplify_path_brute(path):
    # repeatedly find '..' and remove it plus the preceding
    # component via string search/slicing -- fragile, O(n^2)
    # in the worst case; not shown in full, see optimal ins
    return None
```

3. Complexity: O(n²) time, O(n) space

4. Bottleneck: Raw string surgery for 'go up a directory' is fragile and repeats backward scans; directory names naturally behave like a stack.

5. Optimize: Split on '/'. Skip empty/'.' pieces, pop on '..' (if non-empty), push real names. Join what remains.

6. Code it (narrate as you write):

```
def simplify_path(path):
    stack = []
    for part in path.split('/'):
        if part == '' or part == '.':
            continue
        elif part == '..':
            if stack:
                stack.pop()
        else:
            stack.append(part)
    return '/' + '/'.join(stack)
```

9. Final complexity: O(n) time, O(n) space

7. Test: '/a/./b/../../c/' -> a,b push then both pop via the two '..', c pushes -> '/c/'. Edge: '.' at the root, e.g. '/./' -> pop guarded, stays at '/'.

8. Other edge cases: repeated slashes, e.g. '/a//b'; already-canonical path; path of only '.' components.

Memory hook: "Split by slash; real names push, '..' pops, '.' does nothing."

Find K Closest Elements

LeetCode #658 | **MEDIUM** | Pattern: Binary Search | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given a sorted array, a target x , and integer k , return the k closest values to x , sorted ascending. Ask: Ties broken toward the smaller value? Output must stay sorted ascending?

2. Brute force: Sort all elements by distance to x , take the first k , then re-sort those by value.

```
def find_closest_elements(arr, k, x):
    arr2 = sorted(arr, key=lambda v: abs(v - x))
    return sorted(arr2[:k])
```

3. Complexity: $O(n \log n)$ time, $O(n)$ space

4. Bottleneck: Sorting the whole array by distance does far more work than needed -- the array's already sorted, so the answer is one contiguous window.

5. Optimize: Binary search for the window's left boundary: compare the element just before the window to the element just past it, and shift toward whichever is closer to x .

6. Code it (narrate as you write):

```
def find_closest_elements(arr, k, x):
    left, right = 0, len(arr) - k
    while left < right:
        mid = (left + right) // 2
        if x - arr[mid] > arr[mid + k] - x:
            left = mid + 1
        else:
            right = mid
    return arr[left:left + k]
```

9. Final complexity: $O(\log(n-k) + k)$ time, $O(k)$ space

7. Test: $[1,2,3,4,5]$, $k=4$, $x=3$ -> search settles on $\text{start}=0$ -> $[1,2,3,4]$. Edge: k equals array length -> search range collapses to one point, returns everything.

8. Other edge cases: x smaller than every element; x larger than every element; $k=1$.

Memory hook: "Sorted array means the answer is one window -- binary search for where it starts."

Split Array Largest Sum

LeetCode #410 | **HARD** | Pattern: Binary Search on the Answer | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Split an array into m contiguous subarrays to minimize the largest subarray sum; return that minimized value. *Ask:* All numbers non-negative? Need the actual split, or just the minimized value?

2. Brute force: Try every placement of $m-1$ dividers and evaluate the resulting max subarray sum.

```
from itertools import combinations

def split_array_brute(nums, m):
    n = len(nums)
    best = float('inf')
    for cuts in combinations(range(1, n), m - 1):
        parts, prev = [], 0
        for c in cuts:
            parts.append(sum(nums[prev:c])); prev = c
        parts.append(sum(nums[prev:]))
        best = min(best, max(parts))
    return best # combinatorial, impractical for larger n
```

3. Complexity: Exponential time, $O(m)$ space

4. Bottleneck: Exploring every split configuration directly is combinatorial; instead, binary search over candidate ANSWER values and check feasibility.

5. Optimize: Binary search the answer between $\max(\text{nums})$ and $\text{sum}(\text{nums})$. For a candidate cap, greedily count subarrays needed to keep every part $\leq$ cap; feasible if count $\leq m$.

6. Code it (narrate as you write):

```
def split_array(nums, m):
    def subarrays_needed(cap):
        count, cur = 1, 0
        for num in nums:
            if cur + num > cap:
                count += 1
                cur = num
            else:
                cur += num
        return count

    left, right = max(nums), sum(nums)
    while left < right:
        mid = (left + right) // 2
        if subarrays_needed(mid) <= m:
            right = mid
        else:
            left = mid + 1
    return left
```

9. Final complexity: $O(n \log(\text{sum-max}))$ time, $O(1)$ space

7. Test: $[7,2,5,10,8]$, $m=2 \rightarrow$ binary search converges to 18 ($[7,2,5]$ and $[10,8]$). Edge: m equals array length $\rightarrow$ answer is just the largest single element.

8. Other edge cases: $m=1$ (answer is the total sum); all elements identical; one very large element dominating.

Memory hook: "Binary search the answer; greedily count subarrays to test feasibility."

Pow(x, n)

LeetCode #50 | **MEDIUM** | Pattern: Fast Exponentiation (Divide & Conquer) | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Implement pow(x, n) for integer n, which can be negative, efficiently. Ask: Can n be negative (return reciprocal)? n=0 always gives 1?

2. Brute force: Multiply x by itself n times in a loop.

```
def my_pow(x, n):
    if n < 0:
        x = 1 / x
        n = -n
    result = 1
    for _ in range(n):
        result *= x
    return result
```

3. Complexity: O(n) time, O(1) space

4. Bottleneck: Adding one factor of x at a time does far more multiplications than necessary -- repeated squaring covers exponentially more ground per step.

5. Optimize: $x^n = (x^{(n//2)})^2$, with one extra factor of x if n is odd. Recursing halves the exponent every step -- logarithmic multiplications instead of linear.

6. Code it (narrate as you write):

```
def my_pow(x, n):
    if n < 0:
        x = 1 / x
        n = -n
    def fast_pow(base, exp):
        if exp == 0:
            return 1
        half = fast_pow(base, exp // 2)
        return half * half if exp % 2 == 0 else half * half * base
    return fast_pow(x, n)
```

9. Final complexity: O(log n) time, O(log n) recursion space

7. Test: $2^{10} \rightarrow$ halves 10,5,2,1,0, squaring back up gives 1024. Edge: negative exponent, e.g. $2^{-2} \rightarrow$ flip to $0.5^2 = 0.25$.

8. Other edge cases: n=0 (always 1); x=0 with positive n; x=1 or x=-1.

Memory hook: "Halve the exponent, square the result -- add one extra factor if odd."

Add Two Numbers

LeetCode #2 | **MEDIUM** | Pattern: Linked List Traversal + Carry | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Two linked lists represent non-negative integers with digits in reverse order; add them and return the sum as a linked list in the same format. Ask: Digits stored least-significant-first, so add directly from the heads? Lists can differ in length?

2. Brute force: Convert both lists to integers, add, convert back (works in Python; not general/robust).

```
def add_two_numbers_brute(l1, l2):
    def to_int(l):
        num, mult = 0, 1
        while l:
            num += l.val * mult
            mult *= 10
            l = l.next
        return num
    total = to_int(l1) + to_int(l2)
    dummy = cur = ListNode()
    if total == 0:
        return ListNode(0)
    while total:
        cur.next = ListNode(total % 10)
        cur = cur.next
        total //= 10
    return dummy.next
```

3. Complexity: $O(n+m)$ time, $O(n+m)$ space

7. Test: 2->4->3 (342) + 5->6->4 (465) -> 7->0->8 (807). Edge: leftover carry after both lists end, e.g. 99+1 -> loop continues once more for the extra digit, giving 100.

8. Other edge cases: both lists represent 0; very different lengths; sum needs a new leading digit.

Memory hook: "Add node by node, carry the tens digit forward, just like paper addition."

4. Bottleneck: Works in Python (big ints), but sidesteps the intended digit-by-digit simulation and isn't safe in languages with fixed-size integers.

5. Optimize: Walk both lists together, summing digits plus carry, creating a new node for (sum mod 10) and carrying (sum // 10) forward.

6. Code it (narrate as you write):

```
def add_two_numbers(l1, l2):
    dummy = cur = ListNode()
    carry = 0
    while l1 or l2 or carry:
        v1 = l1.val if l1 else 0
        v2 = l2.val if l2 else 0
        total = v1 + v2 + carry
        carry = total // 10
        cur.next = ListNode(total % 10)
        cur = cur.next
        l1 = l1.next if l1 else None
        l2 = l2.next if l2 else None
    return dummy.next
```

9. Final complexity: $O(\max(n,m))$ time, $O(\max(n,m))$ space

Linked List Cycle

LeetCode #141 | **EASY** | Pattern: Fast & Slow Pointers | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given the head of a linked list, determine whether it contains a cycle. Ask: Just true/false needed, not the cycle's start node? Single node can point to itself?

2. Brute force: Hash set of visited nodes; a repeat means a cycle.

```
def has_cycle(head):
    seen = set()
    while head:
        if head in seen:
            return True
        seen.add(head)
        head = head.next
    return False
```

3. Complexity: $O(n)$ time, $O(n)$ space

4. Bottleneck: Remembering every visited node costs $O(n)$ memory when two pointers at different speeds can detect a loop with none.

5. Optimize: Slow moves 1 step, fast moves 2. If there's a cycle, fast eventually laps slow and they meet; if not, fast hits null first.

6. Code it (narrate as you write):

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    if slow == fast:
        return True
    return False
```

9. Final complexity: $O(n)$ time, $O(1)$ space

7. Test: A list that loops back into itself -- slow and fast meet after a few steps. Edge: no cycle, e.g. 1->2->3->None -> fast hits null, returns False.

8. Other edge cases: empty list; single node with no self-loop; single node pointing to itself.

Memory hook: "Two runners on a track -- if there's a loop, the fast one always laps the slow one."

Intersection of Two Linked Lists

LeetCode #160 | **EASY** | Pattern: Two Pointers | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given the heads of two singly linked lists, find the node where they intersect (share the same tail), or null. *Ask:* Intersection means same node by reference, not just equal value? Lists can differ in length before merging?

2. Brute force: For every node in A, scan all of B for a reference match.

```
def get_intersection_node(headA, headB):
    a = headA
    while a:
        b = headB
        while b:
            if a is b:
                return a
            b = b.next
        a = a.next
    return None
```

3. Complexity: $O(n * m)$ time, $O(1)$ space

4. Bottleneck: Rescanning list B for every node in A redoes the same comparisons; aligning both lists' remaining lengths lets one synchronized pass work instead.

5. Optimize: Two pointers, one per list; each switches to the OTHER list's head when it hits null. This equalizes total distance traveled, so they meet at the intersection (or both hit null together).

6. Code it (narrate as you write):

```
def get_intersection_node(headA, headB):
    a, b = headA, headB
    while a != b:
        a = a.next if a else headB
        b = b.next if b else headA
    return a
```

9. Final complexity: $O(n + m)$ time, $O(1)$ space

7. Test: A length 5, B length 6, sharing a tail -- both pointers converge on the shared node after each switches once. Edge: no intersection at all -- both become null on the same step.

8. Other edge cases: one list is empty; intersection at the very first node (identical lists); intersection at the very last node only.

Memory hook: "When you hit your own end, switch to the other list -- you'll meet at the join."

Copy List with Random Pointer

LeetCode #138 | **MEDIUM** | Pattern: Hash Map (Clone Mapping) | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Each node has 'next' and 'random' pointers (random can point anywhere or null); deep-copy the entire list. Ask: Random pointer can point to any node including itself, or null? Need a fully independent deep copy?

2. Brute force: There's no simpler correct shortcut -- you need SOME lookup from original to clone; the two-pass hash map approach below is already the standard baseline.

```
# A single naive pass trying to wire pointers immediately
# breaks when random points to a node not yet cloned --
# there's no meaningfully 'slower but simpler' alternative
# here; go straight to the two-pass hash map solution.
```

3. Complexity: N/A time, N/A space

4. Bottleneck: Wiring pointers in one pass fails whenever random points to a node not yet cloned -- need every node cloned first, then a second pass to wire.

5. Optimize: Pass 1: create a copy of every node's value, store original->copy in a map. Pass 2: use the map to set each copy's next/random.

6. Code it (narrate as you write):

```
def copy_random_list(head):
    if not head:
        return None
    old_to_new = {}
    node = head
    while node:
        old_to_new[node] = Node(node.val)
        node = node.next
    node = head
    while node:
        copy = old_to_new[node]
        copy.next = old_to_new.get(node.next)
        copy.random = old_to_new.get(node.random)
        node = node.next
    return old_to_new[head]
```

9. Final complexity: O(n) time, O(n) space

7. Test: A(random->C), B(random->A), C(random->C) -> clones rewired identically via the map. Edge: a node's random points to itself -- map resolves it to its own clone correctly.

8. Other edge cases: empty list; single node with random=null; every random pointer set to null.

Memory hook: "Map every original to its copy first, then rewire using the map."

Same Tree

LeetCode #100 | **EASY** | Pattern: DFS Recursion | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given the roots of two binary trees, determine whether they're structurally identical with the same node values. *Ask:* Same means identical structure AND identical values everywhere? Either tree could be empty?

2. Brute force: Serialize both trees (with null markers) and compare the serialized strings.

```
def is_same_tree_brute(p, q):
    def serialize(node):
        if not node:
            return '#'
        return f'{node.val},{serialize(node.left)},{serialize(node.right)}'
    return serialize(p) == serialize(q)
```

3. Complexity: $O(n)$ time, $O(n)$ space

4. Bottleneck: Building full serialized strings is extra work when the two trees can be compared directly node by node with early exit on any mismatch.

5. Optimize: Compare roots: both null matches, one null mismatches, otherwise compare values and recurse into both children.

6. Code it (narrate as you write):

```
def is_same_tree(p, q):
    if p is None and q is None:
        return True
    if p is None or q is None:
        return False
    if p.val != q.val:
        return False
    return is_same_tree(p.left, q.left) and is_same_tree(p.right, q.right)
```

9. Final complexity: $O(\min(n,m))$ time, $O(\min(h1,h2))$ space

7. Test: Two identical trees, root 1 with children 2,3 -> True. Edge: one tree has an extra node the other lacks -- null-vs-non-null check catches it, False.

8. Other edge cases: both trees empty; identical structure but one differing value; single-node trees.

Memory hook: "Match the roots, then recursively match both subtree pairs."

Validate Binary Search Tree

LeetCode #98 | **MEDIUM** | Pattern: DFS with Valid Range | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Determine whether a binary tree is a valid BST (every node strictly between its ancestors' bounds). Ask: Duplicate values allowed? Compare against ALL ancestors, not just the immediate parent?

2. Brute force: Naive (and WRONG) shortcut: only check each node against its immediate children.

```
def is_valid_bst_wrong(node):
    if not node:
        return True
    if node.left and node.left.val >= node.val:
        return False
    if node.right and node.right.val <= node.val:
        return False
    return (is_valid_bst_wrong(node.left) and
            is_valid_bst_wrong(node.right))
# BUG: misses violations from non-immediate ancestors!
```

3. Complexity: O(n) but INCORRECT time, O(h) space

4. Bottleneck: Checking only immediate parent-child pairs misses violations from grandparents or higher ancestors -- the BST property constrains a WHOLE subtree, not just one edge.

5. Optimize: Pass down a valid (low, high) range. Going left tightens the upper bound to the current value; going right tightens the lower bound.

6. Code it (narrate as you write):

```
def is_valid_bst(root):
    def validate(node, low, high):
        if node is None:
            return True
        if not (low < node.val < high):
            return False
        return (validate(node.left, low, node.val) and
                validate(node.right, node.val, high))
    return validate(root, float('-inf'), float('inf'))
```

9. Final complexity: O(n) time, O(h) space

7. Test: 5 with left 3 (right child 6) -> 6 violates 5's left-subtree upper bound even though it's > its direct parent 3. Edge: duplicate value, e.g. node equal to parent -- strict inequality rejects it.

8. Other edge cases: empty tree (valid); single node; deeply skewed valid BST.

Memory hook: "Every node lives inside a shrinking range passed down from its ancestors."

Kth Smallest Element in a BST

LeetCode #230 | **MEDIUM** | Pattern: In-order Traversal | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given a BST and integer k, find the kth smallest value in the tree. Ask: k guaranteed valid (1..node count)? All values unique?

2. Brute force: Full in-order traversal collecting every value, then index k-1.

```
def kth_smallest_brute(root, k):
    values = []
    def inorder(node):
        if node:
            inorder(node.left)
            values.append(node.val)
            inorder(node.right)
    inorder(root)
    return values[k - 1]
```

3. Complexity: O(n) time, O(n) space

4. Bottleneck: Collecting every value even when k is small does unnecessary work and uses unnecessary memory past the kth value.

5. Optimize: Iterative in-order traversal with an explicit stack; stop the instant the kth node is popped, instead of visiting the whole tree.

6. Code it (narrate as you write):

```
def kth_smallest(root, k):
    stack = []
    node = root
    count = 0
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        count += 1
        if count == k:
            return node.val
        node = node.right
```

9. Final complexity: O(h + k) time, O(h) space

7. Test: root=3, left=1(right=2), right=4, k=1 -> walks left to 1, returns immediately. Edge: k = total node count -> must traverse entire tree, returns the max.

8. Other edge cases: single-node tree with k=1; fully left/right-skewed tree; k in the middle of a balanced tree.

Memory hook: "In-order traversal is sorted order -- just stop counting at k."

Construct Binary Tree from Preorder and Inorder Traversal

LeetCode #105 | **MEDIUM** | Pattern: Recursion + Hash Map | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given preorder and inorder traversal arrays of a tree with unique values, reconstruct and return the tree. Ask: All node values guaranteed unique? Both arrays same length, valid traversals of the same tree?

2. Brute force: Same recursive idea, but linear-search inorder for each root's split point every call.

```
def build_tree_brute(preorder, inorder):
    if not preorder:
        return None
    root_val = preorder[0]
    root = TreeNode(root_val)
    mid = inorder.index(root_val) # O(n) search every call
    root.left = build_tree_brute(preorder[1:mid+1], inorder[:mid])
    root.right = build_tree_brute(preorder[mid+1:], inorder[mid+1:])
    return root
```

3. Complexity: $O(n^2)$ time, $O(n)$ space

4. Bottleneck: Searching inorder from scratch at every recursive call redoes lookups; precomputing all positions once removes that repeated search.

5. Optimize: Precompute value->index in inorder ($O(1)$ lookup). Track a running preorder pointer; build left subtree first (consumes exactly its share of preorder), then right.

6. Code it (narrate as you write):

```
def build_tree(preorder, inorder):
    inorder_index = {val: i for i, val in enumerate(inorder)}
    self_idx = 0
    def build(left, right):
        nonlocal self_idx
        if left > right:
            return None
        root_val = preorder[self_idx]
        self_idx += 1
        root = TreeNode(root_val)
        mid = inorder_index[root_val]
        root.left = build(left, mid - 1)
        root.right = build(mid + 1, right)
        return root
    return build(0, len(inorder) - 1)
```

9. Final complexity: $O(n)$ time, $O(n)$ space

7. Test: preorder=[3,9,20,15,7], inorder=[9,3,15,20,7] -> root 3, left 9, right 20(left 15, right 7). Edge: empty arrays -> initial range invalid, returns None.

8. Other edge cases: single-node tree; completely left-skewed tree; completely right-skewed tree.

Memory hook: "Preorder's first value is root; inorder's position of that value splits left from right."

Serialize and Deserialize Binary Tree

LeetCode #297 | **HARD** | Pattern: DFS Preorder + Null Markers | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Design an algorithm to serialize a binary tree to a string and deserialize it back to the original tree. Ask: Any string format acceptable as long as it round-trips exactly? Values could contain the delimiter?

2. Brute force: A traversal that OMMITS null markers loses structural information and can't be reliably reversed.

```
def serialize_lossy(root):
    values = []
    def dfs(node):
        if node:
            values.append(str(node.val))
            dfs(node.left)
            dfs(node.right)
    dfs(root)
    return ','.join(values) # ambiguous: can't tell where
```

3. Complexity: O(n) but lossy time, O(n) space

4. Bottleneck: Without null markers, different tree shapes can produce the identical serialized sequence -- deserialization becomes ambiguous or impossible.

5. Optimize: Preorder DFS, writing '#' for every null child. Deserialize by reading tokens in the same preorder order with an iterator.

6. Code it (narrate as you write):

```
def serialize(root):
    vals = []
    def dfs(node):
        if node is None:
            vals.append('#')
            return
        vals.append(str(node.val))
        dfs(node.left); dfs(node.right)
    dfs(root)
    return ','.join(vals)

def deserialize(data):
    vals = iter(data.split(','))
    def build():
        val = next(vals)
        if val == '#':
            return None
        node = TreeNode(int(val))
        node.left = build()
        node.right = build()
        return node
    return build()
```

9. Final complexity: O(n) time, O(n) space

7. Test: root=1,left=2,right=3(left=4,right=5) -> '1,2,##,3,4,##,5,##' round-trips exactly. Edge: empty tree -> serializes to just '#', deserializes to None.

8. Other edge cases: single-node tree; fully left-skewed tree; values with multiple digits or negative signs.

Memory hook: "Preorder plus explicit nulls -- nothing is ambiguous."

Balanced Binary Tree

LeetCode #110 | **EASY** | Pattern: Bottom-Up DFS | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Determine whether a binary tree is height-balanced: for every node, its left/right subtree heights differ by at most 1. Ask: Balance must hold at EVERY node, not just the root? Empty tree counts as balanced?

2. Brute force: At every node, independently recompute the height of its left and right subtrees from scratch.

```
def is_balanced_brute(root):
    def height(node):
        if not node:
            return 0
        return 1 + max(height(node.left), height(node.right))
    if not root:
        return True
    if abs(height(root.left) - height(root.right)) > 1:
        return False
    return is_balanced_brute(root.left) and is_balanced_brute(root.right)
```

3. Complexity: $O(n^2)$ time, $O(h)$ space

4. Bottleneck: Recomputing height from scratch at every node re-walks overlapping subtrees repeatedly.

5. Optimize: One recursive pass returns real height, or a sentinel -1 the moment any subtree below is already unbalanced, short-circuiting further checks.

6. Code it (narrate as you write):

```
def is_balanced(root):
    def check(node):
        if node is None:
            return 0
        left = check(node.left)
        if left == -1:
            return -1
        right = check(node.right)
        if right == -1:
            return -1
        if abs(left - right) > 1:
            return -1
        return 1 + max(left, right)
    return check(root) != -1
```

9. Final complexity: $O(n)$ time, $O(h)$ space

7. Test: root=3, left=9, right=20(15,7) -> every level within 1, height 3, balanced. Edge: imbalance buried deep -- the -1 sentinel bubbles all the way up correctly.

8. Other edge cases: empty tree; single node; imbalance right at the root.

Memory hook: "Compute height and check balance together; -1 means 'already broken, stop caring'."

Subtree of Another Tree

LeetCode #572 | **EASY** | Pattern: DFS Tree Matching | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given roots of two trees, determine whether subRoot exactly matches some subtree of root. Ask: Whole subtree at the matching node must match exactly, including all descendants?

2. Brute force: Same idea as optimal but naively re-derives same-tree logic inline every call without reuse -- functionally identical, just less clean.

```
def is_subtree_brute(root, sub_root):
    def same(a, b):
        if not a and not b:
            return True
        if not a or not b or a.val != b.val:
            return False
        return same(a.left, b.left) and same(a.right, b.right)
    if not root:
        return False
    if same(root, sub_root):
        return True
    return (is_subtree_brute(root.left, sub_root) or
            is_subtree_brute(root.right, sub_root))
# This IS roughly the optimal approach -- there is no
# meaningfully slower correct baseline beyond this shape
```

3. Complexity: $O(n * m)$ time, $O(h_1 + h_2)$ space

4. Bottleneck: This problem doesn't have a deeper asymptotic shortcut for typical interview scope; the win is writing the same-tree check cleanly and trying it at every node.

5. Optimize: Reuse a same-tree helper; call it at every node of root, recursing into children if the current node doesn't match.

6. Code it (narrate as you write):

```
def is_same_tree(a, b):
    if a is None and b is None:
        return True
    if a is None or b is None or a.val != b.val:
        return False
    return is_same_tree(a.left, b.left) and is_same_tree(a.right, b.right)

def is_subtree(root, sub_root):
    if root is None:
        return False
    if is_same_tree(root, sub_root):
        return True
    return is_subtree(root.left, sub_root) or is_subtree(root.right, sub_root)
```

9. Final complexity: $O(n * m)$ time, $O(h_1 + h_2)$ space

7. Test: root=3(left=4(1,2), right=5), subRoot=4(1,2) -> root doesn't match, left child does -> True. Edge: matching value but different subtree shape -- same-tree check rejects it, correctly keeps searching.

8. Other edge cases: subRoot is a single leaf; root equals subRoot exactly; subRoot never appears (False).

Memory hook: "Try an exact match at every node -- reuse Same Tree as the check."

Walls and Gates

LeetCode #286 | **MEDIUM** | Pattern: Multi-Source BFS | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Fill each empty room in a grid with its distance to the nearest gate (0); walls (-1) unchanged. Ask: 4-directional movement only? Multiple gates possible? Modify the grid in place?

2. Brute force: Run a separate BFS from every empty room out to find its nearest gate.

```
from collections import deque

def walls_and_gates_brute(rooms):
    rows, cols = len(rooms), len(rooms[0])
    def bfs_from(sr, sc):
        visited = {(sr, sc)}
        q = deque([(sr, sc, 0)])
        while q:
            r, c, d = q.popleft()
            if rooms[r][c] == 0 and (r, c) != (sr, sc):
                return d
            for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
                nr, nc = r+dr, c+dc
                if 0<=nr<rows and 0<=nc<cols and (nr,nc) not in visited:
                    visited.add((nr,nc)); q.append((nr,nc,d+1))
        return rooms[sr][sc]
    for r in range(rows):
        for c in range(cols):
            if rooms[r][c] not in (0, -1):
                rooms[r][c] = bfs_from(r, c)
```

3. Complexity: $O((rows*cols)^2)$ time, $O(rows*cols)$ space

7. Test: Grid with 2 gates -> BFS expands outward from both simultaneously, filling correct distances. Edge: a room walled off from every gate -- never reached, stays at the sentinel (unreachable).

8. Other edge cases: single gate; no gates at all (everything stays sentinel); grid that's entirely walls except one gate.

Memory hook: "Start BFS from every gate at once -- first arrival is always the nearest."

4. Bottleneck: A full grid-sized BFS from every room redoes overlapping searches; starting from all gates at once visits every cell only once total.

5. Optimize: Seed the BFS queue with every gate simultaneously. BFS explores in increasing distance order, so the first time a room is reached is via its true nearest gate.

6. Code it (narrate as you write):

```
from collections import deque

def walls_and_gates(rooms):
    rows, cols = len(rooms), len(rooms[0])
    queue = deque((r, c) for r in range(rows) for c in range(cols) if rooms[r][c] == 0)
    dirs = [(-1,0), (1,0), (0,-1), (0,1)]
    while queue:
        r, c = queue.popleft()
        for dr, dc in dirs:
            nr, nc = r+dr, c+dc
            if 0<=nr<rows and 0<=nc<cols and rooms[nr][nc] == 2147483647:
                rooms[nr][nc] = rooms[r][c] + 1
                queue.append((nr, nc))
```

9. Final complexity: $O(rows*cols)$ time, $O(rows*cols)$ space

Kth Largest Element in a Stream

LeetCode #703 | **EASY** | Pattern: Heap (size-k min-heap) | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Design a class that supports adding numbers to a stream and always returning the kth largest so far. Ask: k stays fixed for the object's lifetime? 'add' called many times, so must be efficient per call?

2. Brute force: Store every number in a list; each add, re-sort and pick the kth from the end.

```
class KthLargestBrute:
    def __init__(self, k, nums):
        self.k = k
        self.nums = list(nums)
    def add(self, val):
        self.nums.append(val)
        self.nums.sort()
        return self.nums[-self.k]
```

3. Complexity: $O(n \log n)$ per add time, $O(n)$ space

4. Bottleneck: Re-sorting the entire history on every call redoes work; only the top k values ever matter, not the full history's order.

5. Optimize: Keep a min-heap of exactly the k largest values seen. The heap's top (smallest of the top-k) is always the answer; push and pop only when a new value beats it.

6. Code it (narrate as you write):

```
import heapq

class KthLargest:
    def __init__(self, k, nums):
        self.k = k
        self.heap = []
        for num in nums:
            self.add(num)
    def add(self, val):
        if len(self.heap) < self.k:
            heapq.heappush(self.heap, val)
        elif val > self.heap[0]:
            heapq.heapreplace(self.heap, val)
        return self.heap[0]
```

9. Final complexity: $O(\log k)$ per add time, $O(k)$ space

7. Test: $k=3$, add 4,5,8,2 -> heap settles on {4,5,8}, kth largest=4. Edge: adding a value smaller than the current min of a full heap -- ignored, heap unchanged.

8. Other edge cases: $k=1$ (track just the max); duplicate values; initial nums array is empty.

Memory hook: "Min-heap of size k; the top of the heap IS the kth largest."

Jump Game

LeetCode #55 | **MEDIUM** | Pattern: Greedy | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given max jump lengths per index starting at 0, determine if you can reach the last index. Ask: Jump lengths can be 0 (potential trap)? Just yes/no needed, not the actual path?

2. Brute force: Recursively try every possible jump length from each position.

```
def can_jump_brute(nums):
    def dfs(i):
        if i >= len(nums) - 1:
            return True
        for step in range(1, nums[i] + 1):
            if dfs(i + step):
                return True
        return False
    return dfs(0)
```

3. Complexity: $O(2^n)$ time, $O(n)$ space

4. Bottleneck: Exploring every specific jump path is unnecessary -- only the single furthest reachable index matters, not which exact route got there.

5. Optimize: Scan left to right tracking furthest reachable. If the current index is beyond that, you're stuck; otherwise update $\text{furthest} = \max(\text{furthest}, i + \text{nums}[i])$.

6. Code it (narrate as you write):

```
def can_jump(nums):
    furthest = 0
    for i in range(len(nums)):
        if i > furthest:
            return False
        furthest = max(furthest, i + nums[i])
    return True
```

9. Final complexity: $O(n)$ time, $O(1)$ space

7. Test: [2,3,1,1,4] -> furthest grows to cover index 4, True. Edge: a trapping 0, e.g. [3,2,1,0,4] -> furthest stalls at 3, index 4 unreachable, False.

8. Other edge cases: single-element array (trivially True); array starting with 0 (only True if length 1); long stretch of zeros.

Memory hook: "Track the furthest you could ever reach -- forget the actual path."

Task Scheduler

LeetCode #621 | **MEDIUM** | Pattern: Greedy + Math | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given tasks and a cooldown n between repeats of the same task, find the minimum total time (with idle slots) to finish all tasks. Ask: Only the SAME task needs the cooldown gap? Idle time counts toward the returned total?

2. Brute force: Simulate minute by minute with a max-heap, always picking the most frequent available task.

```
import heapq
from collections import Counter

def least_interval_brute(tasks, n):
    counts = list(Counter(tasks).values())
    heap = [-c for c in counts]
    heapq.heapify(heap)
    time = 0
    q = [] # (available_time, count)
    while heap or q:
        time += 1
        if heap:
            c = heapq.heappop(heap) + 1
            if c < 0:
                q.append((time + n, c))
            if q and q[0][0] == time:
                heapq.heappush(heap, q.pop(0)[1])
    return time
```

3. Complexity: $O(t \log 26)$ time, $O(1)$ space

7. Test: A,A,A,B,B,B with $n=2$ -> both tie for $\text{max_freq}=3$, $\text{formula}=(3-1)*3+2=8$, matches A-B-idle-A-B-idle-A-B. Edge: $n=0$ (no cooldown) -> $\text{max}()$ resolves to just $\text{len}(\text{tasks})$.

8. Other edge cases: one task type with large n (lots of idle); many distinct single-occurrence tasks (no idle needed); all tasks identical.

Memory hook: "The busiest task sets the pace -- formula beats simulating minute by minute."

4. Bottleneck: Simulating every single minute works but is more computation than necessary; the schedule's length is fully determined by counting, not simulating.

5. Optimize: Let max_freq be the highest count and max_count how many tasks tie for it. $\text{Schedule} = (\text{max_freq}-1)$ full $(n+1)$ -wide chunks + a final chunk of max_count tasks -- unless enough OTHER tasks fill every gap, in which case the answer is just $\text{len}(\text{tasks})$.

6. Code it (narrate as you write):

```
from collections import Counter

def least_interval(tasks, n):
    counts = Counter(tasks)
    max_freq = max(counts.values())
    max_count = sum(1 for c in counts.values() if c == max_freq)
    formula = (max_freq - 1) * (n + 1) + max_count
    return max(formula, len(tasks))
```

9. Final complexity: $O(t)$ time, $O(1)$ space

Car Pooling

LeetCode #1094 | **MEDIUM** | Pattern: Difference Array / Line Sweep | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given trips [passengers, start, end] and a car capacity, determine if all trips can be completed without exceeding capacity. Ask: Drop-off at a location frees capacity before pick-up at that same location? Locations bounded (e.g. 0-1000)?

2. Brute force: For every location along the route, sum passengers from all currently-overlapping trips directly.

```
def car_pooling_brute(trips, capacity):
    max_loc = max(end for _, _, end in trips)
    for loc in range(max_loc):
        total = sum(p for p, s, e in trips if s <= loc < e)
        if total > capacity:
            return False
    return True
```

3. Complexity: $O(\text{trips} * \text{max_location})$ time, $O(1)$ space

4. Bottleneck: Recomputing the total at every single location redoes the same overlap checks; passenger count only actually changes at trip start/end points.

5. Optimize: Build a difference array: add passengers at start, subtract at end. Sweep once with a running sum, checking against capacity as you go.

6. Code it (narrate as you write):

```
def car_pooling(trips, capacity):
    changes = [0] * 1001
    for passengers, start, end in trips:
        changes[start] += passengers
        changes[end] -= passengers
    current = 0
    for change in changes:
        current += change
        if current > capacity:
            return False
    return True
```

9. Final complexity: $O(\text{trips} + \text{max_location})$ time, $O(\text{max_location})$ space

7. Test: `[[2,1,5],[3,3,7]]`, capacity=4 -> at location 3, both trips overlap = 5 passengers > 4, False. Edge: a drop-off and pick-up at the exact same location -- both apply at that index, correctly freeing capacity first.

8. Other edge cases: a single trip exactly filling capacity; non-overlapping trips; a trip with 0 passengers.

Memory hook: "Add at pickup, subtract at drop-off, sweep and watch the running total."

Generate Parentheses

LeetCode #22 | **MEDIUM** | Pattern: Backtracking | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given n , generate all combinations of n pairs of well-formed (balanced) parentheses. *Ask:* Order of the output doesn't matter? n guaranteed small (output size grows fast)?

2. Brute force: Generate every string of length $2n$, then filter for validity.

```
def generate_parenthesis_brute(n):
    def is_valid(s):
        bal = 0
        for ch in s:
            bal += 1 if ch == '(' else -1
            if bal < 0:
                return False
        return bal == 0
    result = []
    def gen(s):
        if len(s) == 2 * n:
            if is_valid(s):
                result.append(s)
            return
        gen(s + '(')
        gen(s + ')')
    gen('')
    return result
```

3. Complexity: $O(2^{2n} * n)$ time, $O(n)$ space

4. Bottleneck: Generating and then filtering wastes huge effort on strings that are invalid from an early character -- pruning invalid choices immediately avoids that entirely.

5. Optimize: Only add '(' if under n opens used; only add ')' if it wouldn't exceed the opens placed so far. Every full-length string built this way is automatically valid.

6. Code it (narrate as you write):

```
def generate_parenthesis(n):
    result = []
    def backtrack(current, opens, closes):
        if len(current) == 2 * n:
            result.append(current)
            return
        if opens < n:
            backtrack(current + '(', opens + 1, closes)
        if closes < opens:
            backtrack(current + ')', opens, closes + 1)
    backtrack('', 0, 0)
    return result
```

9. Final complexity: $O(4^n / \sqrt{n})$ time, $O(4^n / \sqrt{n})$ space

7. Test: $n=2$ -> produces '()' and '()()'. Edge: $n=0$ -> immediately returns [''] (empty string).

8. Other edge cases: $n=1$ (just '()'); verifying output count matches the n th Catalan number; ensuring no invalid string like ')(' is ever produced.

Memory hook: "Only add ')' if there's an unmatched '(' waiting for it."

Combination Sum

LeetCode #39 | **MEDIUM** | Pattern: Backtracking | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given distinct positive candidates and a target, find all unique combinations summing to target, reusing numbers freely. Ask: Same number can be reused unlimited times? Combos with same numbers in different order count once?

2. Brute force: Explore every sequence of chosen numbers without pruning or an ordering constraint.

```
def combination_sum_brute(candidates, target):
    result = []
    def dfs(path, remaining):
        if remaining == 0:
            result.append(list(path))
            return
        if remaining < 0:
            return
        for c in candidates: # no start index -> duplicate
            path.append(c)
            dfs(path, remaining - c)
            path.pop()
    dfs([], target)
    return result # produces duplicate permutations of the
```

3. Complexity: Exponential w/ duplicates time, $O(\text{target}/\min)$ space

4. Bottleneck: Allowing any candidate at every step (not just forward) generates the same combination multiple times in different orders, wasting work and needing de-duplication.

5. Optimize: Only consider candidates from the current index onward (never backward); this guarantees each combination is found in exactly one canonical order.

6. Code it (narrate as you write):

```
def combination_sum(candidates, target):
    result = []
    def backtrack(start, path, remaining):
        if remaining == 0:
            result.append(list(path))
            return
        if remaining < 0:
            return
        for i in range(start, len(candidates)):
            path.append(candidates[i])
            backtrack(i, path, remaining - candidates[i])
            path.pop()
    backtrack(0, [], target)
    return result
```

9. Final complexity: Exponential (pruned) time, $O(\text{target}/\min)$ space

7. Test: [2,3,6,7], target=7 -> finds [2,2,3] and [7]. Edge: no combination reaches target, e.g. candidates=[5], target=3 -> immediately overshoots, empty result.

8. Other edge cases: target equals one candidate exactly; a candidate that alone exceeds target (pruned); many repetitions of the smallest candidate.

Memory hook: "Only move forward through candidates; reuse allowed by staying at the same index."

Letter Combinations of a Phone Number

LeetCode #17 | **MEDIUM** | Pattern: Backtracking | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given a digit string (2-9), return all possible letter combinations per the phone keypad mapping. Ask: Standard keypad mapping? Empty input should return an empty list?

2. Brute force: Fixed nested loops only work for a KNOWN number of digits -- doesn't generalize; would need one loop written per possible length.

```
def letter_combinations_fixed_3(digits):
    # Only works for exactly 3 digits -- illustrates why a
    # fixed number of nested loops doesn't generalize.
    phone = {'2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
            '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'}
    result = []
    for a in phone[digits[0]]:
        for b in phone[digits[1]]:
            for c in phone[digits[2]]:
                result.append(a + b + c)
    return result
```

3. Complexity: $O(4^n)$ time, $O(n)$ space

4. Bottleneck: Nested loops require knowing the digit count in advance; recursion naturally handles any input length by processing one digit per level.

5. Optimize: Recursively try every letter for the current digit, advance to the next digit; once the combination's length matches the digit count, record it.

6. Code it (narrate as you write):

```
def letter_combinations(digits):
    if not digits:
        return []
    phone = {'2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
            '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'}
    result = []
    def backtrack(i, current):
        if i == len(digits):
            result.append(current)
            return
        for ch in phone[digits[i]]:
            backtrack(i + 1, current + ch)
    backtrack(0, '')
    return result
```

9. Final complexity: $O(4^n * n)$ time, $O(n)$ space

7. Test: '23' -> pairs a,b,c with d,e,f -> 9 combos, ad through cf. Edge: empty string -> guard clause returns [] immediately.

8. Other edge cases: single digit; a digit mapping to 4 letters (7 or 9); repeated identical digits like '22'.

Memory hook: "One recursive level per digit -- try every letter, then move to the next digit."

Valid Sudoku

LeetCode #36 | **MEDIUM** | Pattern: Hash Set Constraint Checking | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Determine whether a 9x9 Sudoku board's current configuration is valid (no repeated digit in any row, column, or 3x3 box). Ask: Only checking current validity, not solvability? Empty cells always '.'

2. Brute force: Check all rows, then all columns, then all boxes in three separate full passes.

```
def is_valid_sudoku_brute(board):
    def has_dup(cells):
        vals = [c for c in cells if c != '.']
        return len(vals) != len(set(vals))
    for row in board:
        if has_dup(row):
            return False
    for c in range(9):
        if has_dup([board[r][c] for r in range(9)]):
            return False
    for br in range(0, 9, 3):
        for bc in range(0, 9, 3):
            cells = [board[r][c] for r in range(br, br+3) for c in range(bc, bc+3) if board[r][c] != '.']
            if has_dup(cells):
                return False
    return True
```

3. Complexity: O(1) (fixed 81 cells), 3 passes time, O(1) space

4. Bottleneck: Three separate full passes visit every cell three times total; all three constraints can be checked for each cell in a single pass instead.

5. Optimize: Keep 9 sets each for rows, columns, boxes. For each filled cell, check/add to all three relevant sets in one scan; box index = $(r//3)*3 + (c//3)$.

6. Code it (narrate as you write):

```
def is_valid_sudoku(board):
    rows = [set() for _ in range(9)]
    cols = [set() for _ in range(9)]
    boxes = [set() for _ in range(9)]
    for r in range(9):
        for c in range(9):
            val = board[r][c]
            if val == '.':
                continue
            b = (r // 3) * 3 + (c // 3)
            if val in rows[r] or val in cols[c] or val in boxes[b]:
                return False
            rows[r].add(val); cols[c].add(val); boxes[b].add(val)
    return True
```

9. Final complexity: O(1) fixed board time, O(1) space

7. Test: Digit '5' repeated in the same row -> second occurrence caught immediately. Edge: duplicate only within the same 3x3 box (different row/col) -- box index calculation catches it.

8. Other edge cases: completely empty board (valid); a fully correct solved board (valid); only one digit filled in anywhere.

Memory hook: "One pass, three sets per cell: row, column, and box."

Climbing Stairs

LeetCode #70 | **EASY** | Pattern: DP (Fibonacci) | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: A staircase has n steps; each move climbs 1 or 2 steps. Count the distinct ways to reach the top. Ask: Only 1 or 2 steps per move, never more? Different orderings count as distinct ways?

2. Brute force: Plain recursion: $\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2)$, no caching.

```
def climb_stairs_brute(n):
    if n <= 2:
        return n
    return climb_stairs_brute(n - 1) + climb_stairs_brute(n - 2)
```

3. Complexity: $O(2^n)$ time, $O(n)$ space

4. Bottleneck: Recomputing the same smaller subproblems across different branches wastes exponential work; each value only needs to be computed once.

5. Optimize: This is exactly Fibonacci. Build up from the bottom with two rolling variables instead of full recursion or even a full array.

6. Code it (narrate as you write):

```
def climb_stairs(n):
    if n <= 2:
        return n
    prev2, prev1 = 1, 2
    for _ in range(3, n + 1):
        prev2, prev1 = prev1, prev1 + prev2
    return prev1
```

9. Final complexity: $O(n)$ time, $O(1)$ space

7. Test: $n=5 \rightarrow$ sequence 1,2,3,5,8 $\rightarrow$ answer 8. Edge: $n=1 \rightarrow$ base case returns 1 directly.

8. Other edge cases: $n=2$ (exactly 2 ways); $n=0$ if it occurs (edge convention); large n verifying no recursion blowup.

Memory hook: "It's just Fibonacci -- today's ways is yesterday's plus the day before."

Perfect Squares

LeetCode #279 | **MEDIUM** | Pattern: DP (Unbounded Knapsack style) | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given n , find the minimum number of perfect squares (1,4,9,...) that sum exactly to n . Ask: Same square can be reused multiple times? Is there always a valid answer (yes, via 1's)?

2. Brute force: Recursively try subtracting every possible square, no caching.

```
def num_squares_brute(n):
    if n == 0:
        return 0
    best = float('inf')
    j = 1
    while j * j <= n:
        best = min(best, 1 + num_squares_brute(n - j * j))
        j += 1
    return best
```

3. Complexity: Exponential time, $O(n)$ space

4. Bottleneck: Recomputing the same remainder values across different subtraction paths wastes massive redundant work.

5. Optimize: $dp[i]$ = min squares summing to i . For each i , try every square $j*j \leq i$: $dp[i] = \min(dp[i], dp[i-j*j] + 1)$, building bottom-up.

6. Code it (narrate as you write):

```
def num_squares(n):
    dp = [float('inf')] * (n + 1)
    dp[0] = 0
    for i in range(1, n + 1):
        j = 1
        while j * j <= i:
            dp[i] = min(dp[i], dp[i - j*j] + 1)
            j += 1
    return dp[n]
```

9. Final complexity: $O(n * \sqrt{n})$ time, $O(n)$ space

7. Test: $n=12 \rightarrow$ dp builds up to 3 (4+4+4). Edge: n is itself a perfect square, e.g. $16 \rightarrow dp[16]=1$.

8. Other edge cases: $n=1$ (trivially 1); a number needing the theoretical max of 4 squares; a prime not near any perfect square.

Memory hook: "Coin change, but the coins are perfect squares."

Is Subsequence

LeetCode #392 | **EASY** | Pattern: Two Pointers (Greedy Matching) | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given strings *s* and *t*, determine if *s* is a subsequence of *t* (same relative order, not necessarily contiguous). Ask: Characters must stay in relative order but need not be contiguous? Case-sensitive comparison?

2. Brute force: There's no meaningfully slower correct alternative beyond generating subsequences of *t*, which is exponential and impractical -- the greedy scan below is already the natural approach.

```
def is_subsequence_brute(s, t):
    from itertools import combinations
    if not s:
        return True
    for combo in combinations(range(len(t)), len(s)):
        if all(t[combo[i]] == s[i] for i in range(len(s))):
            return True
    return False # exponential, illustrative only
```

3. Complexity: $O(2^{\text{len}(t)})$ time, $O(2^{\text{len}(t)})$ space

4. Bottleneck: Explicitly considering subsequences of *t* is exponential; matching greedily left to right needs only one linear pass.

5. Optimize: Scan *t* once; whenever the current character matches what *s* still needs next, advance *s*'s pointer. *s* is a subsequence iff that pointer reaches the end.

6. Code it (narrate as you write):

```
def is_subsequence(s, t):
    i = 0
    for ch in t:
        if i < len(s) and ch == s[i]:
            i += 1
    return i == len(s)
```

9. Final complexity: $O(\text{len}(t))$ time, $O(1)$ space

7. Test: *s*='abc', *t*='ahbgdc' -> matches a,b,c in order, True. Edge: *s* longer than *t*, e.g. *s*='abc', *t*='ab' -> *t* runs out first, False.

8. Other edge cases: *s* is empty (trivially True); *s* equals *t*; *t* contains all of *s*'s letters but in the wrong order.

Memory hook: "Greedly match each letter of *s* to the next occurrence in *t*."

Distinct Subsequences

LeetCode #115 | **HARD** | Pattern: 2D DP (Counting) | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Count how many distinct ways t appears as a subsequence of s (different position choices count separately). *Ask:* Different position choices spelling the same substring count as separate ways? t longer than s means 0?

2. Brute force: Recursively try, for each character of t , every possible matching position in the rest of s .

```
def num_distinct_brute(s, t):
    def dfs(i, j):
        if j == len(t):
            return 1
        if i == len(s):
            return 0
        count = dfs(i + 1, j) # skip s[i]
        if s[i] == t[j]:
            count += dfs(i + 1, j + 1) # use s[i]
        return count
    return dfs(0, 0)
```

3. Complexity: Exponential time, $O(n+m)$ space

4. Bottleneck: The same (remaining s , remaining t) subproblems recur across many branches without caching.

5. Optimize: $dp[i][j]$ = ways to form $t[j]$ using $s[i]$. Always carry forward $dp[i-1][j]$ (skip); if characters match, also add $dp[i-1][j-1]$ (use).

6. Code it (narrate as you write):

```
def num_distinct(s, t):
    m, n = len(s), len(t)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = 1
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            dp[i][j] = dp[i-1][j]
            if s[i-1] == t[j-1]:
                dp[i][j] += dp[i-1][j-1]
    return dp[m][n]
```

9. Final complexity: $O(m*n)$ time, $O(m*n)$ space

7. Test: $s='rabbbit'$, $t='rabbit'$ -> classic answer is 3. Edge: t longer than s , e.g. $s='ab'$, $t='abc'$ -> final cell stays 0.

8. Other edge cases: t is empty (1 way, choose nothing); s equals t (1 way); s has many repeated characters relevant to t .

Memory hook: "On a match, add BOTH 'use it' and 'skip it' -- it's counting, not choosing."

Unique Binary Search Trees

LeetCode #96 | **MEDIUM** | Pattern: DP (Catalan Numbers) | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given n , count how many structurally unique BSTs can be formed using values $1..n$. Ask: Just count distinct STRUCTURES, not enumerate/build the actual trees?

2. Brute force: Recursively try every value as root, recursing on left/right ranges without caching by size.

```
def num_trees_brute(n):
    def count(lo, hi):
        if lo >= hi:
            return 1
        total = 0
        for root in range(lo, hi + 1):
            total += count(lo, root - 1) * count(root + 1, hi)
        return total
    return count(1, n)
```

3. Complexity: Exponential-ish (uncached) time, $O(n)$ space

4. Bottleneck: The count for a range only depends on its SIZE, not its specific values, so the same size gets recomputed repeatedly across different ranges.

5. Optimize: $dp[i]$ = unique BSTs using i values in a row. For each size i , sum $dp[k-1] * dp[i-k]$ over every root position k .

6. Code it (narrate as you write):

```
def num_trees(n):
    dp = [0] * (n + 1)
    dp[0] = dp[1] = 1
    for i in range(2, n + 1):
        for k in range(1, i + 1):
            dp[i] += dp[k - 1] * dp[i - k]
    return dp[n]
```

9. Final complexity: $O(n^2)$ time, $O(n)$ space

7. Test: $n=3 \rightarrow$ dp builds 1,1,2,5 $\rightarrow$ answer 5. Edge: $n=0 \rightarrow dp[0]=1$ base case (one empty-tree shape).

8. Other edge cases: $n=1$ (1 shape); $n=2$ (2 shapes); verifying the sequence matches known Catalan numbers.

Memory hook: "Fix a root; multiply left-shape count by right-shape count; sum over every root choice."

Insert Delete GetRandom O(1)

LeetCode #380 | **MEDIUM** | Pattern: Hash Map + Array | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Design a structure supporting insert, remove, and getRandom on a set of unique values, all average O(1). Ask: All values unique (a true set)? insert/remove should return whether something actually changed?

2. Brute force: Plain list: insert checks for duplicates and appends; remove searches for the value's position and shifts.

```
class RandomizedSetBrute:
    def __init__(self):
        self.values = []
    def insert(self, val):
        if val in self.values:
            return False
        self.values.append(val)
        return True
    def remove(self, val):
        if val not in self.values:
            return False
        self.values.remove(val) # O(n) search + shift
        return True
    def get_random(self):
        import random
        return random.choice(self.values)
```

3. Complexity: insert/remove O(n) time, O(n) space

4. Bottleneck: A list alone can't do O(1) lookup/removal by value; a hash map alone can't do O(1) random access -- need both, plus a trick to make removal O(1) too.

5. Optimize: Array holds values (O(1) random access); hash map holds value->index (O(1) lookup). To remove in O(1): swap the target with the array's LAST element, update the map, then pop the end.

6. Code it (narrate as you write):

```
import random

class RandomizedSet:
    def __init__(self):
        self.values = []
        self.index = {}
    def insert(self, val):
        if val in self.index:
            return False
        self.index[val] = len(self.values)
        self.values.append(val)
        return True
    def remove(self, val):
        if val not in self.index:
            return False
        i = self.index[val]
        last = self.values[-1]
        self.values[i] = last
        self.index[last] = i
        self.values.pop()
        del self.index[val]
        return True
    def get_random(self):
        return random.choice(self.values)
```

9. Final complexity: O(1) average time, O(n) space

7. Test: insert(1),insert(2),insert(3),remove(2) -> 3 swaps into 2's slot, map updates, last slot pops -> values=[1,3]. Edge: removing the value that's already last -- swap-with-self is harmless, still pops correctly.

8. Other edge cases: removing a nonexistent value (False, no change); inserting a duplicate (False); getRandom with only one element.

Memory hook: "Array for random access, hash map for lookup, swap-with-last to remove."

Reverse Bits

LeetCode #190 | **EASY** | Pattern: Bit Manipulation | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Reverse the bits of a 32-bit unsigned integer and return the resulting integer. Ask: Always exactly 32 bits, zero-padded? Output also treated as 32-bit unsigned?

2. Brute force: Convert to a 32-character binary string, reverse the string, parse back to an integer.

```
def reverse_bits_brute(n):
    binary = format(n, '032b')
    reversed_binary = binary[::-1]
    return int(reversed_binary, 2)
```

3. Complexity: $O(1)$ (fixed 32 chars) time, $O(1)$ space

4. Bottleneck: No real asymptotic bottleneck for a fixed 32-bit width; the point is showing direct bit manipulation with shifts/masks instead of string conversion.

5. Optimize: Extract the lowest bit of n , shift the result left to make room, OR the bit in, then shift n right -- 32 times builds the reversed pattern.

6. Code it (narrate as you write):

```
def reverse_bits(n):
    result = 0
    for _ in range(32):
        bit = n & 1
        result = (result << 1) | bit
        n >>= 1
    return result
```

9. Final complexity: $O(1)$ time, $O(1)$ space

7. Test: Small 4-bit illustration: 0b1011 -> processing bit by bit gives 0b1101, the reverse. Edge: all-zero input -> stays 0 throughout.

8. Other edge cases: all-one input (unchanged); single set bit at the front (moves to the back); a palindromic bit pattern (unchanged).

Memory hook: "Peel one bit off the front, tuck it onto the back of the result."

Number of 1 Bits

LeetCode #191 | **EASY** | Pattern: Bit Manipulation (Kernighan's) | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Count how many bits are set to 1 in an unsigned integer's binary representation. Ask: Input treated as a fixed 32-bit unsigned integer?

2. Brute force: Check all 32 bit positions directly.

```
def hamming_weight_brute(n):
    count = 0
    for i in range(32):
        if (n >> i) & 1:
            count += 1
    return count
```

3. Complexity: O(1) (32 fixed checks) time, O(1) space

4. Bottleneck: Checking every position even when many are 0 does more work than needed; a trick can skip straight from one set bit to the next.

5. Optimize: $n \& (n-1)$ always clears the lowest set bit. Repeat and count until n becomes 0 -- exactly as many steps as there are set bits.

6. Code it (narrate as you write):

```
def hamming_weight(n):
    count = 0
    while n != 0:
        n &= (n - 1)
        count += 1
    return count
```

9. Final complexity: O(k) (k = set bits) time, O(1) space

7. Test: $n=11$ (0b1011) -> 3 iterations clear each set bit, count=3. Edge: $n=0$ -> loop condition false immediately, count=0.

8. Other edge cases: a power of 2 (exactly 1 set bit); all bits set for the width (e.g. 0xFFFFFFFF); $n=1$.

Memory hook: " $n \text{ AND } (n-1)$ erases the lowest 1 -- count how many erasures until zero."

Pascal's Triangle

LeetCode #118 | **EASY** | Pattern: Simple DP (row from previous row) | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given numRows, generate the first numRows rows of Pascal's Triangle. Ask: Every row starts and ends with 1? Output is a list of lists?

2. Brute force: Compute each entry independently via the binomial coefficient formula (n choose k), recomputing factorials.

```
from math import comb

def generate_brute(num_rows):
    return [[comb(r, k) for k in range(r + 1)] for r in range(num_rows)]
```

3. Complexity: $O(\text{numRows}^3)$ naive factorial time, $O(\text{numRows}^2)$ space

4. Bottleneck: Computing each entry independently via factorials redoes overlapping work; each row can be built directly and cheaply from the row before it.

5. Optimize: Each row starts/ends with 1; every middle value is the sum of the two values above it in the previous row.

6. Code it (narrate as you write):

```
def generate(num_rows):
    triangle = [[1]]
    for _ in range(1, num_rows):
        prev = triangle[-1]
        row = [1]
        for i in range(1, len(prev)):
            row.append(prev[i-1] + prev[i])
        row.append(1)
        triangle.append(row)
    return triangle
```

9. Final complexity: $O(\text{numRows}^2)$ time, $O(\text{numRows}^2)$ space

7. Test: numRows=5 -> builds [1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]. Edge: numRows=1 -> stays as just [[1]].

8. Other edge cases: numRows=2; confirming outer values are always 1; numRows=0 (empty list, if allowed).

Memory hook: "Each new row: start with 1, sum adjacent pairs above, end with 1."

Longest Common Prefix

LeetCode #14 | **EASY** | Pattern: Vertical Scanning | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given an array of strings, find the longest common prefix shared by all of them, or "" if none. Ask: Case-sensitive? Can the array or an individual string be empty?

2. Brute force: Compare every pair of strings for their pairwise common prefix, take the shortest across all pairs.

```
def longest_common_prefix_brute(strs):
    def common(a, b):
        i = 0
        while i < len(a) and i < len(b) and a[i] == b[i]:
            i += 1
        return a[:i]
    prefix = strs[0]
    for s in strs[1:]:
        prefix = common(prefix, s)
    return prefix # actually fine, but framed pairwise --
```

3. Complexity: $O(n * m)$ time, $O(1)$ space

4. Bottleneck: Pairwise comparison works but reduces the whole set two at a time; checking all strings together at each position stops at the very first disagreement directly.

5. Optimize: For each character position in the first string, check if every OTHER string agrees there; stop at the first mismatch or short string.

6. Code it (narrate as you write):

```
def longest_common_prefix(strs):
    if not strs:
        return ''
    for i in range(len(strs[0])):
        ch = strs[0][i]
        for s in strs[1:]:
            if i >= len(s) or s[i] != ch:
                return strs[0][:i]
    return strs[0]
```

9. Final complexity: $O(n * m)$ time, $O(1)$ space

7. Test: ["flower", "flow", "flight"] -> matches 'fl', mismatches at position 2. Edge: no common prefix at all, e.g. ["dog", "racecar", "car"] -> mismatch at position 0, returns "".

8. Other edge cases: single string in the array; one string is empty; all strings identical.

Memory hook: "Check one position at a time across every string -- stop at first disagreement."

Integer to Roman

LeetCode #12 | **MEDIUM** | Pattern: Greedy | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Convert an integer to its Roman numeral representation, including subtractive notation (IV, IX, etc.). *Ask:* Input guaranteed within the standard range (1-3999)?

2. Brute force: Hardcode if/elif branches for every possible digit (0-9) at every place value (ones, tens, hundreds, thousands).

```
def int_to_roman_brute(num):
    # Illustrative only: this requires ~40 explicit branches
    # one per digit value per place, e.g.:
    thousands = ['', 'M', 'MM', 'MMM']
    hundreds = ['', 'C', 'CC', 'CCC', 'CD', 'D', 'DC', 'DCC', 'DCCC', 'CM']
    tens = ['', 'X', 'XX', 'XXX', 'XL', 'L', 'LX', 'LXX', 'LXXX', 'XC']
    ones = ['', 'I', 'II', 'III', 'IV', 'V', 'VI', 'VII', 'VIII', 'IX']
    return (thousands[num // 1000] + hundreds[(num // 100) % 10] +
            tens[(num // 10) % 10] + ones[num % 10])
```

3. Complexity: O(1) time, O(1) space

4. Bottleneck: Hardcoding every digit-place combination works but is repetitive and error-prone; a single ordered value list captures the same logic far more concisely.

5. Optimize: An ordered list of (value, symbol) pairs, largest to smallest, explicitly including subtractive combos (900='CM', 4='IV', etc). Greedily subtract the largest fitting value repeatedly.

6. Code it (narrate as you write):

```
def int_to_roman(num):
    values = [(1000, 'M'), (900, 'CM'), (500, 'D'), (400, 'CD'),
              (100, 'C'), (90, 'XC'), (50, 'L'), (40, 'XL'),
              (10, 'X'), (9, 'IX'), (5, 'V'), (4, 'IV'), (1, 'I')]
    result = []
    for value, symbol in values:
        while num >= value:
            result.append(symbol)
            num -= value
    return ''.join(result)
```

9. Final complexity: O(1) time, O(1) space

7. Test: 1994 -> M, CM, XC, IV -> 'MCMXCIV'. Edge: several subtractive combos in a row, e.g. 444 -> CD, XL, IV -> 'CDXLIV'.

8. Other edge cases: smallest input, num=1 ('I'); largest typical input, 3999; a round number like 1000 or 500 needing no subtractive parts.

Memory hook: "Ordered list of values (including CM, CD, XC...) -- always take the biggest bite that fits."

Add Binary

LeetCode #67 | **EASY** | Pattern: Two Pointers + Carry | Not in the linked YouTube playlist — carried over from the original problem list you gave before the playlist link. No video source for this one.

1. Restate & clarify: Given two binary strings, add them and return the sum, also as a binary string. Ask: Strings can be different lengths? Only '0'/'1' characters guaranteed?

2. Brute force: Convert both to integers, add, convert back to binary (works in Python; not general to all languages).

```
def add_binary_brute(a, b):  
    return bin(int(a, 2) + int(b, 2))[2:]
```

3. Complexity: $O(n+m)$ time, $O(n+m)$ space

4. Bottleneck: Works in Python (arbitrary-precision ints) but sidesteps demonstrating the intended digit-by-digit simulation, and isn't safe in fixed-integer-size languages.

5. Optimize: Process both strings right to left with two pointers, summing digits plus carry, building the result and finishing with any leftover carry.

6. Code it (narrate as you write):

```
def add_binary(a, b):  
    result = []  
    i, j = len(a) - 1, len(b) - 1  
    carry = 0  
    while i >= 0 or j >= 0 or carry:  
        da = int(a[i]) if i >= 0 else 0  
        db = int(b[j]) if j >= 0 else 0  
        total = da + db + carry  
        carry = total // 2  
        result.append(str(total % 2))  
        i -= 1; j -= 1  
    return ''.join(reversed(result))
```

9. Final complexity: $O(\max(n,m))$ time, $O(\max(n,m))$ space

7. Test: $a='11'(3)$, $b='1'(1) \rightarrow '100'(4)$. Edge: leftover carry, e.g. $a='1'$, $b='1' \rightarrow$ needs one extra digit, $'10'(2)$.

8. Other edge cases: very different lengths; one string is just '0'; both strings all 1's (maximal carry chain).

Memory hook: "Paper addition in base 2 -- sum digits plus carry, right to left."

Pattern Cheat Sheet

Interview clue	Pattern
Need a pair/complement fast	Hash Map (Two Sum, Contains Duplicate)
Same letters/counts grouped	Hash Map canonical key (Group Anagrams)
Product/sum except self, no division	Prefix * Suffix products
# subarrays summing to k, negatives allowed	Prefix Sum + Hash Map
Palindrome check, O(1) space	Two Pointers from both ends
Maximize area/container between two points	Two Pointers, move the shorter side
Longest substring with a constraint	Sliding Window
Matching brackets / most-recent-first	Stack
Next greater/warmer element to the right	Monotonic Stack
Design O(1) getMin alongside a stack	Auxiliary min-stack
Implement stack with queues (or vice versa)	Two-structure rotation trick
Tree depth / recursive tree property	DFS Recursion
Process a tree/grid level by level	BFS with queue-size snapshot
Deep copy a graph/structure with cycles	DFS/BFS + Hash Map (orig -> clone)
Can all tasks finish given dependencies	Cycle detection / Topological Sort
Sorted array + search	Binary Search
Rotated sorted array	Binary Search vs. right boundary
O(1) get/put + evict least-recently-used	Hash Map + Doubly Linked List
Pick randomly with weighted probability	Prefix Sum + Binary Search
Multiple sources spreading simultaneously	Multi-Source BFS
Max sum of a contiguous subarray	Kadane's Algorithm (DP)
Top K frequent, bound on frequency	Bucket Sort by frequency

Sources

Primary source

The playlist at youtube.com/playlist?list=PL_6hP3Z1GQqYrrF1fETaEpkLAX6M8E7j was provided. This session's network policy blocks youtube.com, and Claude in Chrome is not connected in this cloud session, so none of its videos could be fetched or transcribed. No quotes in this guide are attributed to, or copied from, any video transcript.

Supporting sources

LeetCode problem statements (for exact constraints/return formats) and general, widely-known algorithmic technique names and explanations, written independently.

On the 46 non-playlist problems

46 of the 73 problems in this guide (marked with 'Not in the linked YouTube playlist' under their title) come from the original problem list you gave before the playlist link was shared. They have no associated video at all — not just an untranscribed one.

Videos named in the provided playlist (titles/channels, not transcribed)

- "Two Sum - Leetcode 1 - HashMap - Python" — NeetCode
- "Contains Duplicate - Leetcode 217 - Python" — NeetCode
- "Group Anagrams - Categorize Strings by Count - Leetcode 49" — NeetCode
- "Product of Array Except Self - Leetcode 238 - Python" — NeetCode
- "Subarray Sum Equals K - Prefix Sums - Leetcode 560 - Python" — NeetCode
- "Valid Palindrome - Leetcode 125 - Python" — NeetCode
- "Container with Most Water - Leetcode 11 - Python" — NeetCode
- "Longest Substring Without Repeating Characters - Leetcode 3 - Python" — NeetCode
- "Minimum Window Substring - Airbnb Interview Question - Leetcode 76" — NeetCode
- "Valid Parentheses - Stack - Leetcode 20 - Python" — NeetCode
- "Design Min Stack - Amazon Interview Question - Leetcode 155 - Python" — NeetCode
- "Daily Temperatures - Monotonic Stack - Leetcode 739 - Python" — NeetCode
- "Evaluate Reverse Polish Notation - Leetcode 150 - Python" — NeetCode
- "Implement Queue using Stacks - Leetcode 232 - Python" — NeetCodeIO
- "Implement Stack using Queues - Leetcode 225 - Python" — NeetCode
- "Maximum Depth of Binary Tree - 3 Solutions - Leetcode 104 - Python" — NeetCode
- "Binary Tree Level Order Traversal - BFS - Leetcode 102" — NeetCode
- "Binary Tree Vertical Order Traversal - Leetcode 314 - Python" — NeetCodeIO
- "Clone Graph - Depth First Search - Leetcode 133" — NeetCode
- "Course Schedule - Graph Adjacency List - Leetcode 207" — NeetCode
- "Binary Search - Leetcode 704 - Python" — NeetCode
- "Find Minimum in Rotated Sorted Array - Binary Search - Leetcode 153 - Python" — NeetCode
- "LRU Cache - Twitch Interview Question - Leetcode 146" — NeetCode
- "Random Pick with Weight | Bucket approach | Leetcode #528 | Binary search" — Techdose
- "Rotting Oranges - Leetcode 994 - Python" — NeetCode
- "Maximum Subarray - Amazon Coding Interview Question - Leetcode 53 - Python" — NeetCode
- "Top K Frequent Elements - Bucket Sort - Leetcode 347 - Python" — NeetCode